Week 7

Fine-tuning

Nebius Academy

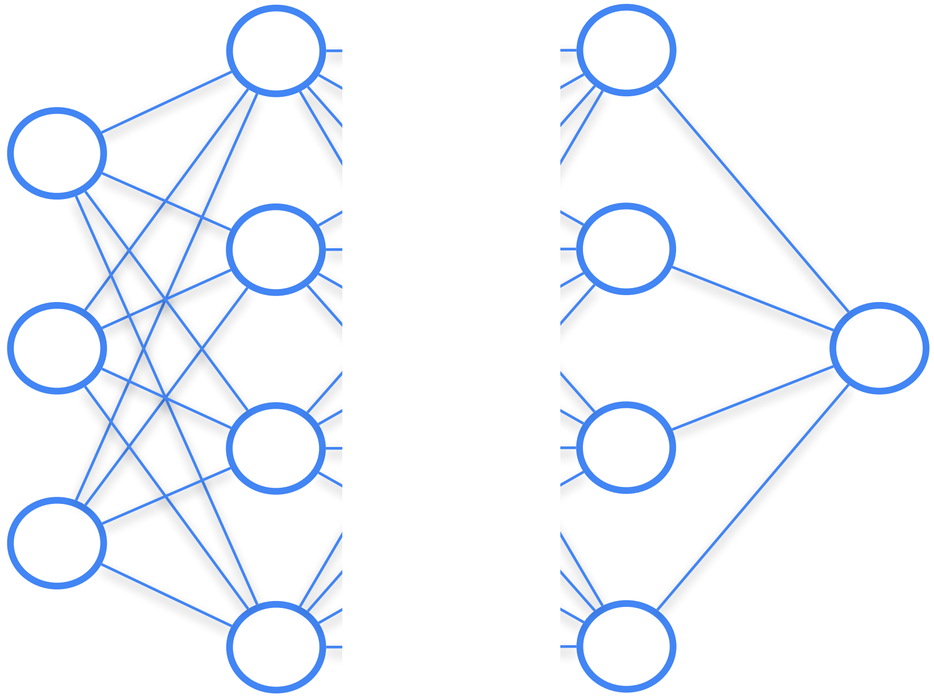
Plan

- Transfer Learning, Fine-Tuning
  - Magic of fine-tuned weights
  - Practice: LoRA
- Representation engineering
- Regularisation
  - SGD with Momentum
  - Dropout
  - BatchNorm

Transfer Learning, Fine-tuning

Features learned by neural network

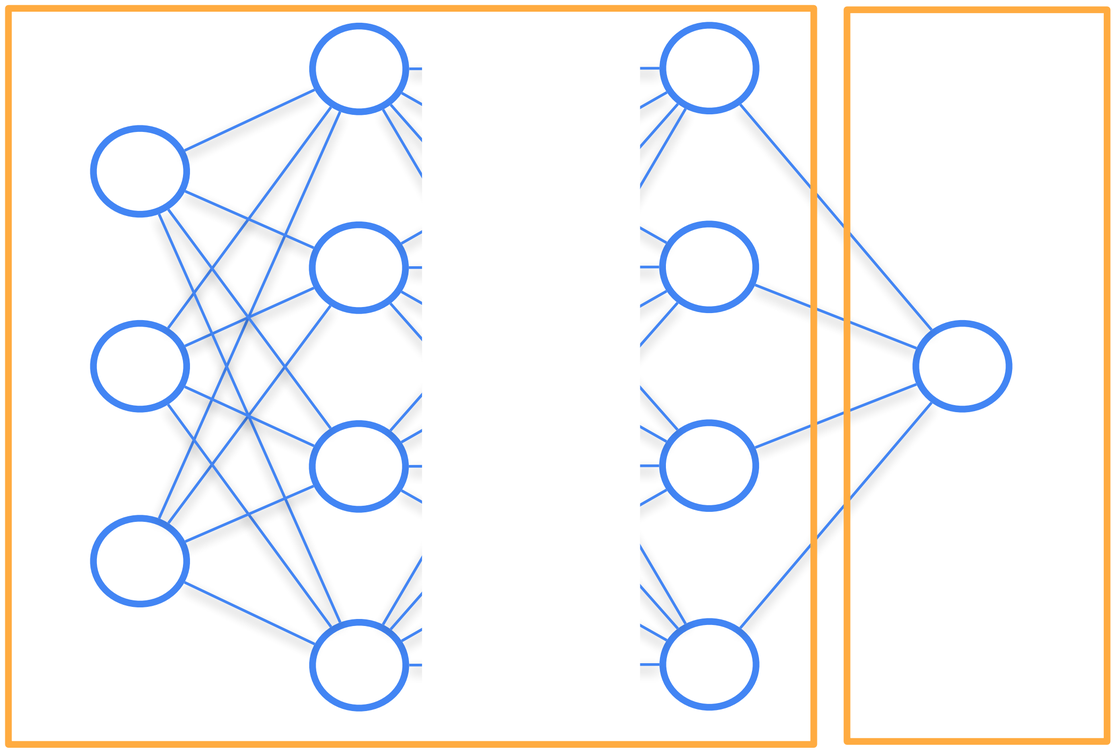
Before we talk about transfer learning, let’s say a couple of words about what types of features different layers in neural network learn



Fully-connected Neural Network

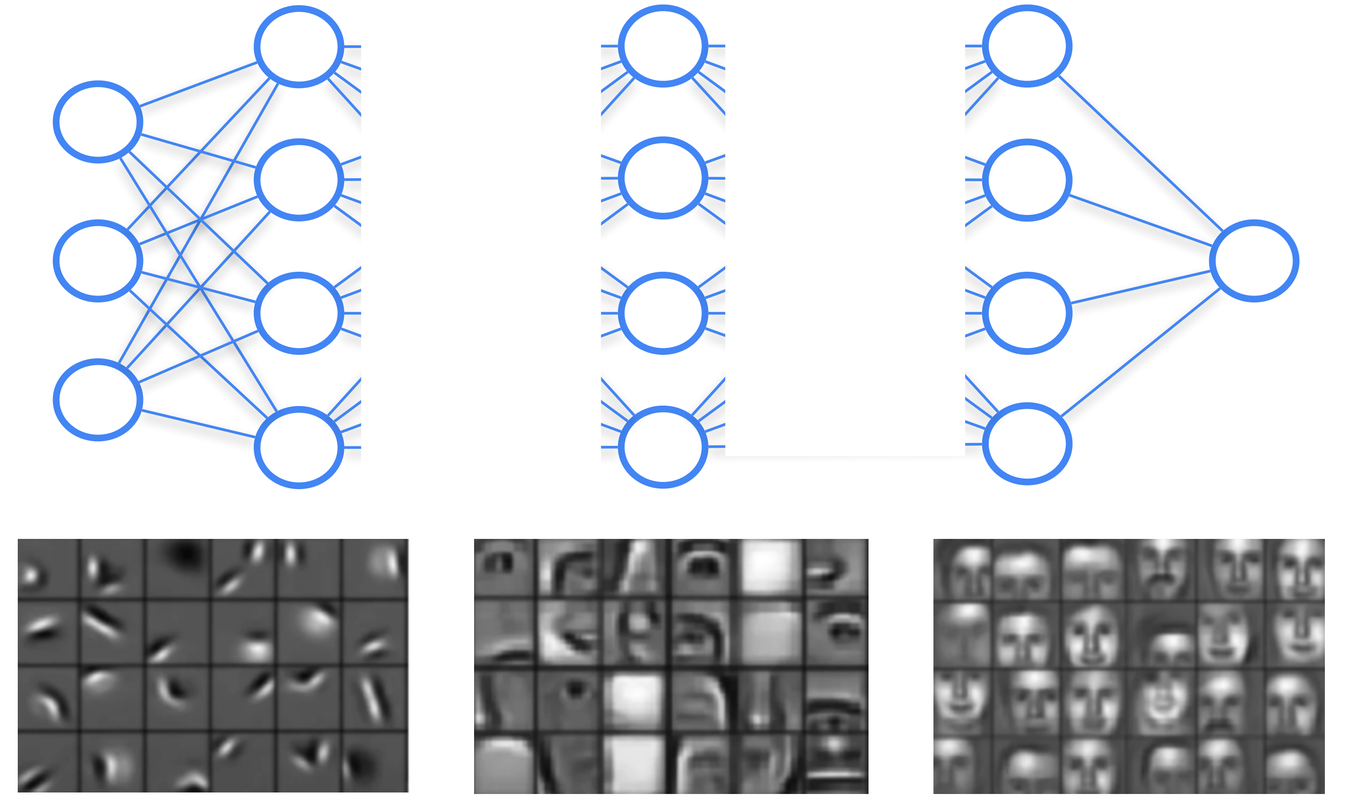
Last layer is usually called classifier,
and all the layers before are called
feature extractors.

In general, layers in the beginning
extract low-level information from data,
and later layers capture more
structured, high-level information.



Features learned in CNNs

What patterns convolutional layers react to:



lower-level features

higher-level features

Transfer Learning

Transfer Learning is a reuse of a (pre-)trained model on a new problem.

The idea is that a pre-trained model has learned useful features which

will help solving new problem.

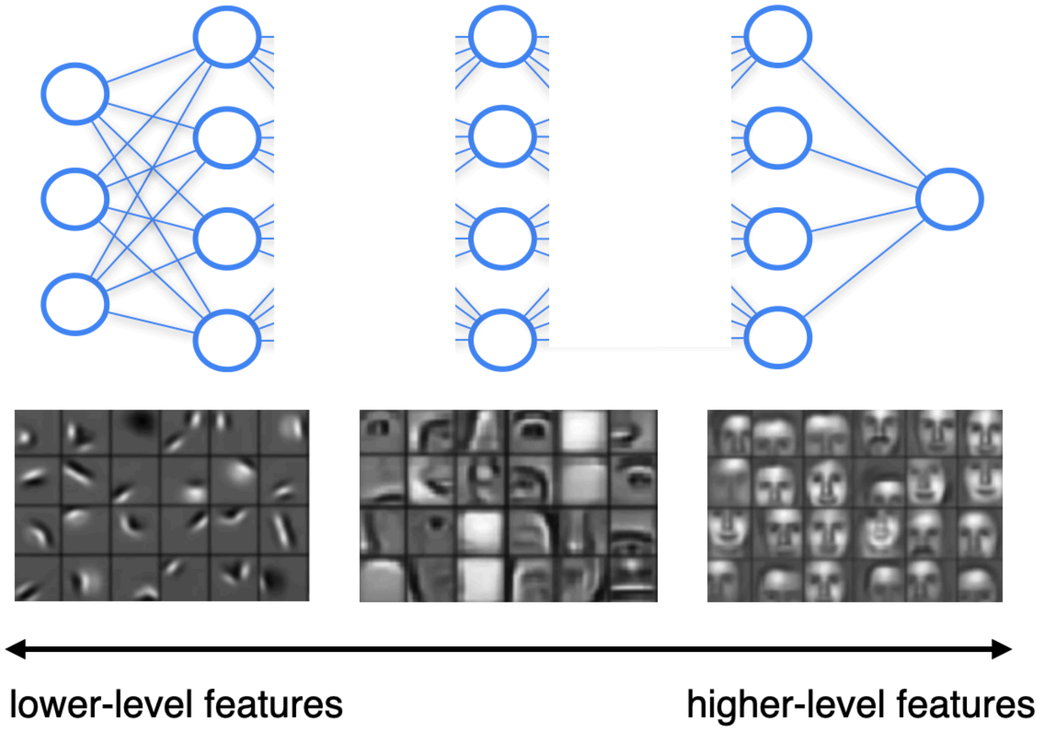
Why not train a model on new data from scratch?

- There is never enough data (and any task can benefit from more data)
- It is costly.

Why transfer learning works

During pre-training, model learns features, that will be useful to solve the final task.

It will need only to learn additional information specific to the new task during fine-tuning



Fine-tuning

Transfer learning is where we train a model on one dataset, then transfer the knowledge to another, related dataset.

The simplest way to do this consists of two steps:

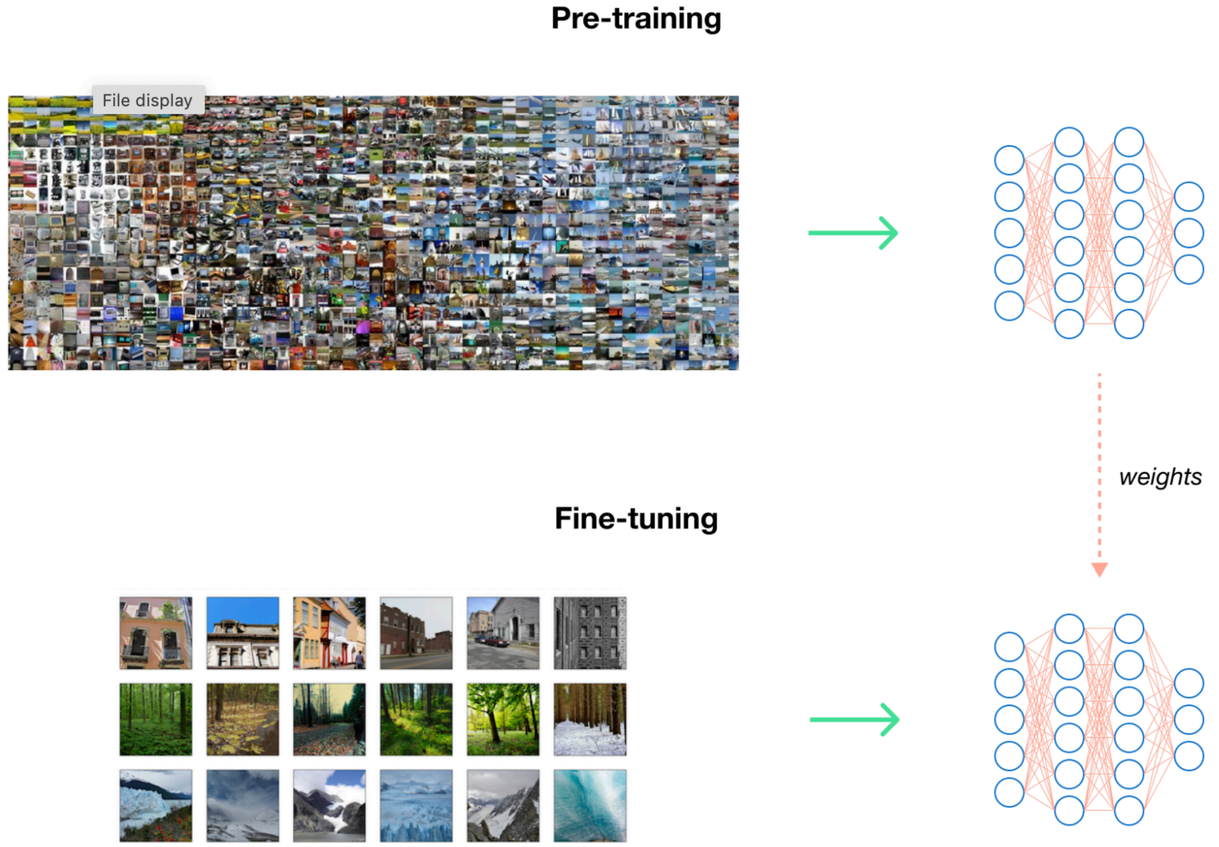
1. **Pre-training** a general model on a big dataset that is only vaguely related to our specific task
2. **Fine-tuning** this model on a smaller dataset for our specific task

Pre-trained LLMs and their instruction tuned/chat versions are good examples of fine-tuning!

Fine tuning how-to

Let’s say we want to train network for scene classification with fine-tuning.

- First, we need a dataset for pre-training. It should be big enough and somewhat relevant to our final task.
- We fine-tune the model on a new, smaller dataset, with smaller number on epochs



Fine tuning how-to

Imagenet is one of the most used datasets for pre-training on image data. It is general-purpose, containing diverse set of images.

https://www.image-net.org/

There are two versions of ImageNet:

- ImageNet-1k, containing ~1.5 million images divided by 1k classes
- ImageNet-21k, containing ~14 million images divided by 21k classes

Fine tuning how-to

ImageNet-1k has 1k classes

If we pre-train on ImageNet, the last layer of network needs to have 1000 neurons

This dataset has fewer classes, let’s say 10. So our network should have 10 neurons in the last layer.

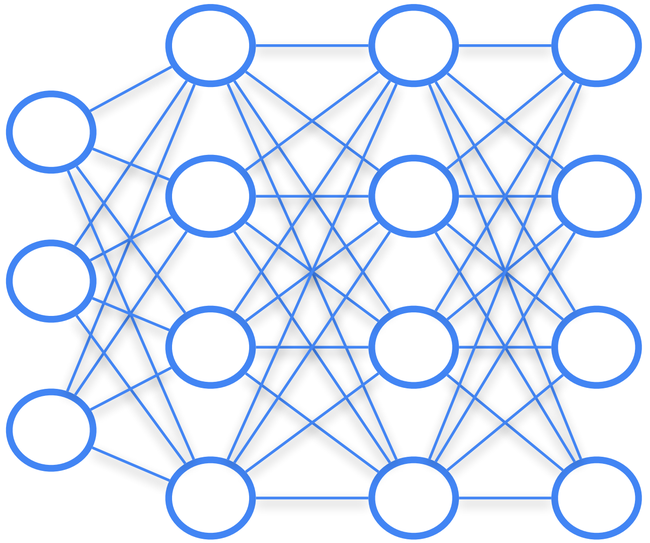
Fine-tuning how-to

Solution:

Before fine-tuning, change the last layer to the new one, containing needed number of neurons

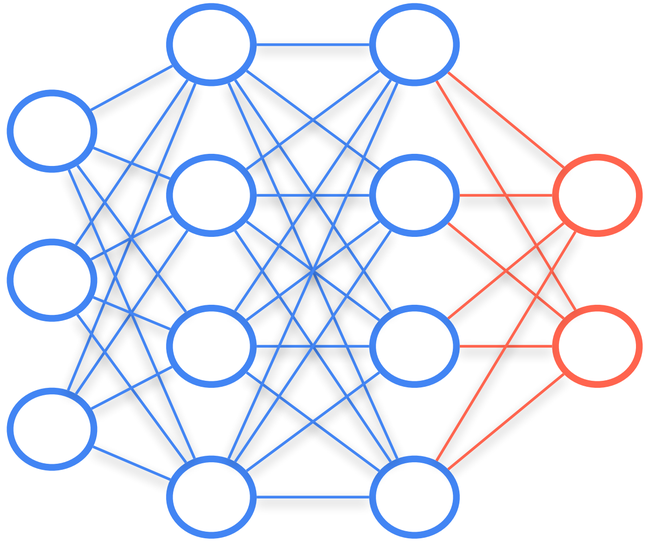
This layer will be trained from scratch during fine-tuning

Pre-training



1000 neurons

Fine-tuning



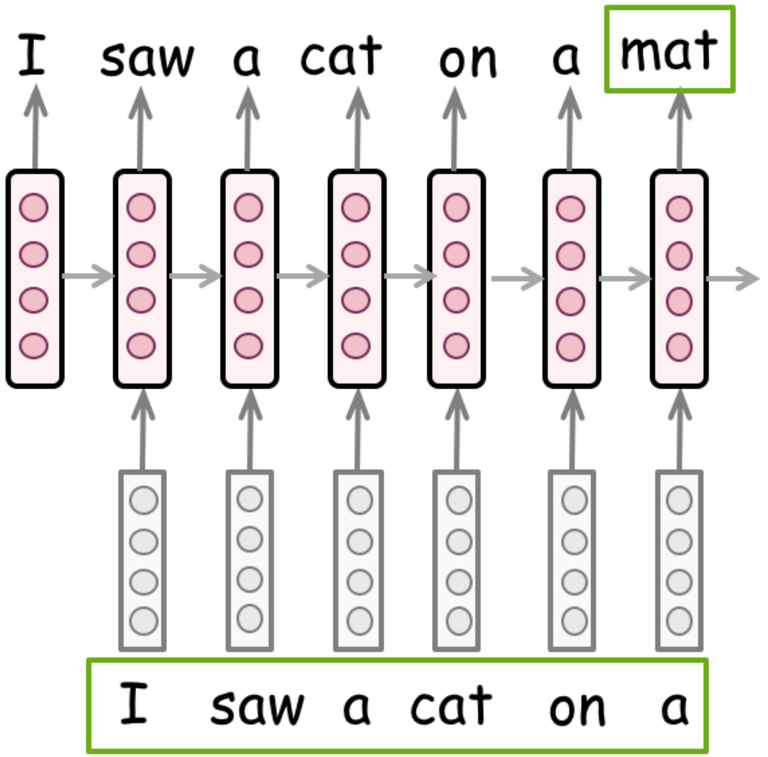
10 neurons

Fine-tuning LLMs

Pre-training of LLMs typically consists of tuning the LLM on next token prediction task on a big amount of natural texts

Fine-tuning usually can be done without changes in architecture. Examples of fine-tuning:

- Texts of different styles
- Instruction following
- Chat capabilities

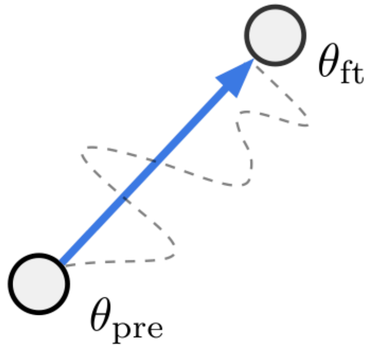


Magic of Fine-Tuning: Task vectors

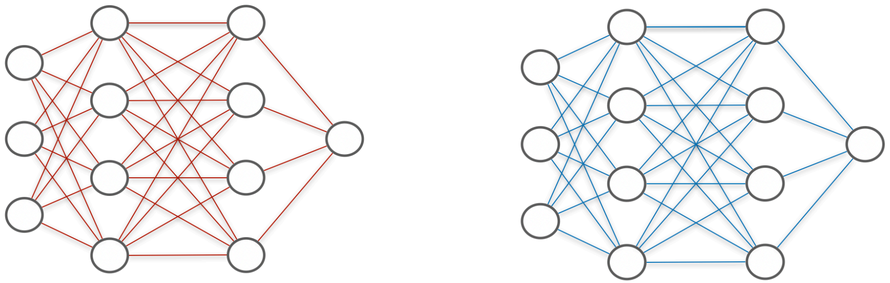
Pre-trained model $M_{pre}\left(X\,|\,\theta_{pre}\right)$

Fine-tuned version on task T_1: $M_{ft}^{T_1}\left(X\,|\,\theta_{ft}^{T_1}\right)$

We can obtain task vector $\tau^{T_1} = \theta_{ft}^{T_1} - \theta_{pre}$



$\tau = \theta_{\text{ft}} - \theta_{\text{pre}}$

$X \rightarrow$  $M_{ft}^{T_1}$ M_{pre}

$\theta_{ft}^{T_1}$ - θ_{pre} = τ^{T_1}

https://arxiv.org/pdf/2212.04089

Magic of Fine-Tuning: Task vectors

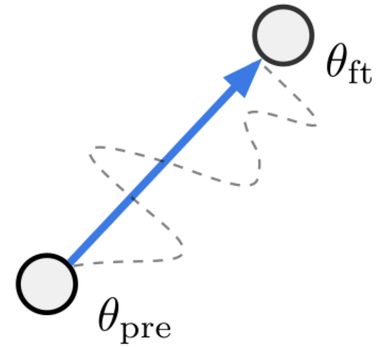
Pre-trained model $M_{pre}\left(X\,|\,\theta_{pre}\right)$

Fine-tuned version on task T_1: $M_{ft}^{T_1}\left(X\,|\,\theta_{ft}^{T_1}\right)$

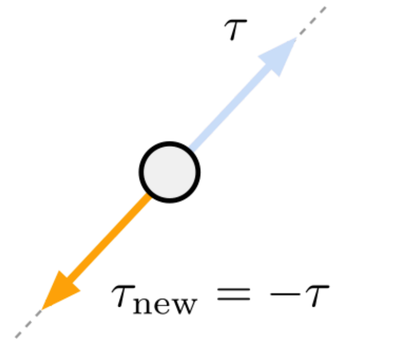
We can obtain task vector $\tau^{T_1} = \theta_{ft}^{T_1} - \theta_{pre}$

Forgetting via negation:

If T_1 is the task of toxicity (e.g. fine-tune GPT-2 to be more toxic), then we can make base model be less toxic via subtracting weight vector $\theta_{pre} - \alpha\tau^{T_1}$



$\tau = \theta_{\text{ft}} - \theta_{\text{pre}}$



Example: making a language model produce less toxic content

https://arxiv.org/pdf/2212.04089

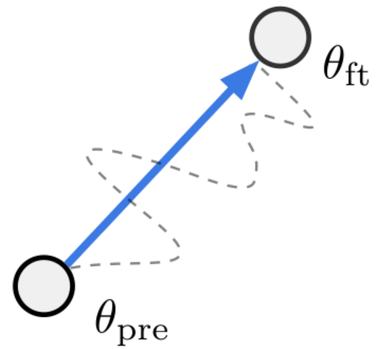
Magic of Fine-Tuning: Task vectors

Table 2: **Making language models less toxic with negative task vectors.** Results are shown for the GPT-2 Large model. Negative task vectors decrease the amount of toxic generations by 6×, while resulting in a model with comparable perplexity on a control task (WikiText-103). Additional details and results are shown in Appendix C.

Method	% toxic generations (↓)	Avg. toxicity score (↓)	WikiText-103 perplexity (↓)
Pre-trained	4.8	0.06	16.4
Fine-tuned	57	0.56	16.6
Gradient ascent	0.0	0.45	$>10^{10}$
Fine-tuned on non-toxic	1.8	0.03	17.2
Random vector	4.8	0.06	16.4
Negative task vector	0.8	0.01	16.9

https://arxiv.org/pdf/2212.04089

Magic of Fine-Tuning: Task vectors



$\tau = \theta_{\text{ft}} - \theta_{\text{pre}}$

Pre-trained model $M_{pre}\left(X \mid \theta_{pre}\right)$

Fine-tuned version on task T_1: $M_{ft}^{T_1}\left(X \mid \theta_{ft}^{T_1}\right)$

We can obtain task vector $\tau^{T_1} = \theta_{ft}^{T_1} - \theta_{pre}$

Improving performance via addition:

If we add task vector τ^{T_1} to $\theta_{ft}^{T_1}$can further improve performance on task T_1

$\theta_{ft}^{T_1} + \alpha\tau^{T_1}$ This is the same as $\theta_{pre} + \alpha\tau^{T_1}$ with $\tau > 1$

https://arxiv.org/pdf/2212.04089

Magic of Fine-Tuning: Task vectors

- Get pre-trained T5 model
- Get several fine-tuned versions, create task vectors
- Add task vectors to pre-trained models

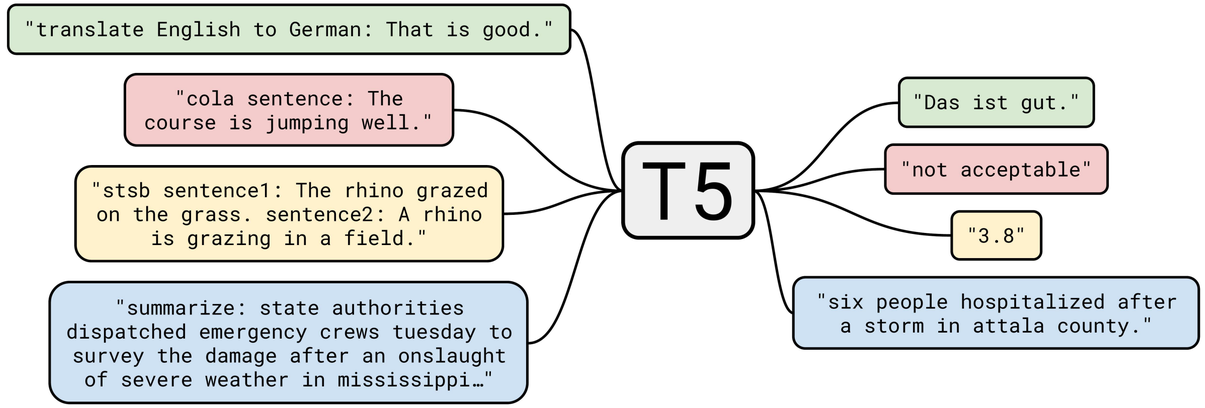


Table 3: **Improving performance on target tasks with external task vectors.** For four text classification tasks from the GLUE benchmark, adding task vectors downloaded from the Hugging Face Hub can improve accuracy of fine-tuned T5 models. Appendix D.6 shows additional details.

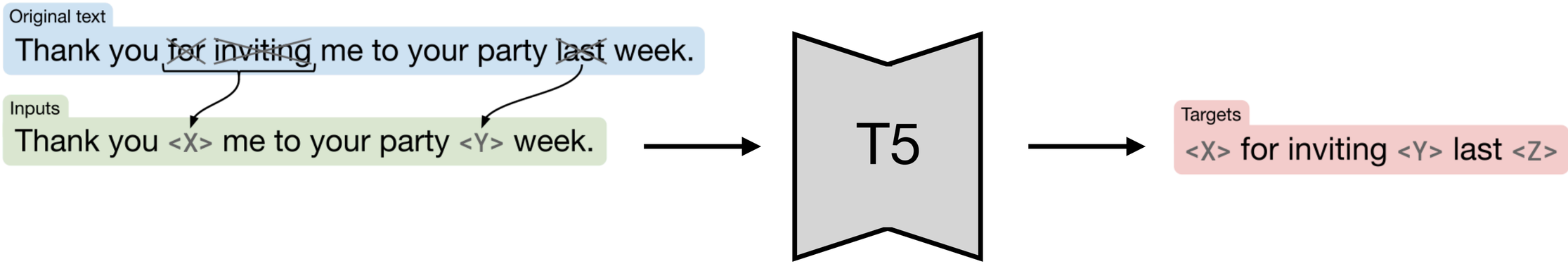
Method	MRPC	RTE	CoLA	SST-2	Average
Zero-shot	74.8	52.7	8.29	92.7	57.1
Fine-tuned	88.5	77.3	52.3	94.5	78.1
Fine-tuned + task vectors	89.3 (+0.8)	77.5 (+0.2)	53.0 (+0.7)	94.7 (+0.2)	78.6 (+0.5)

https://arxiv.org/pdf/2212.04089

T5

T5 is a encoder-decoder text-to-text Transformer model

Pre-training is done via Masked Language Modelling objective



https://arxiv.org/pdf/2212.04089

T5

T5 is a encoder-decoder text-to-text Transformer model

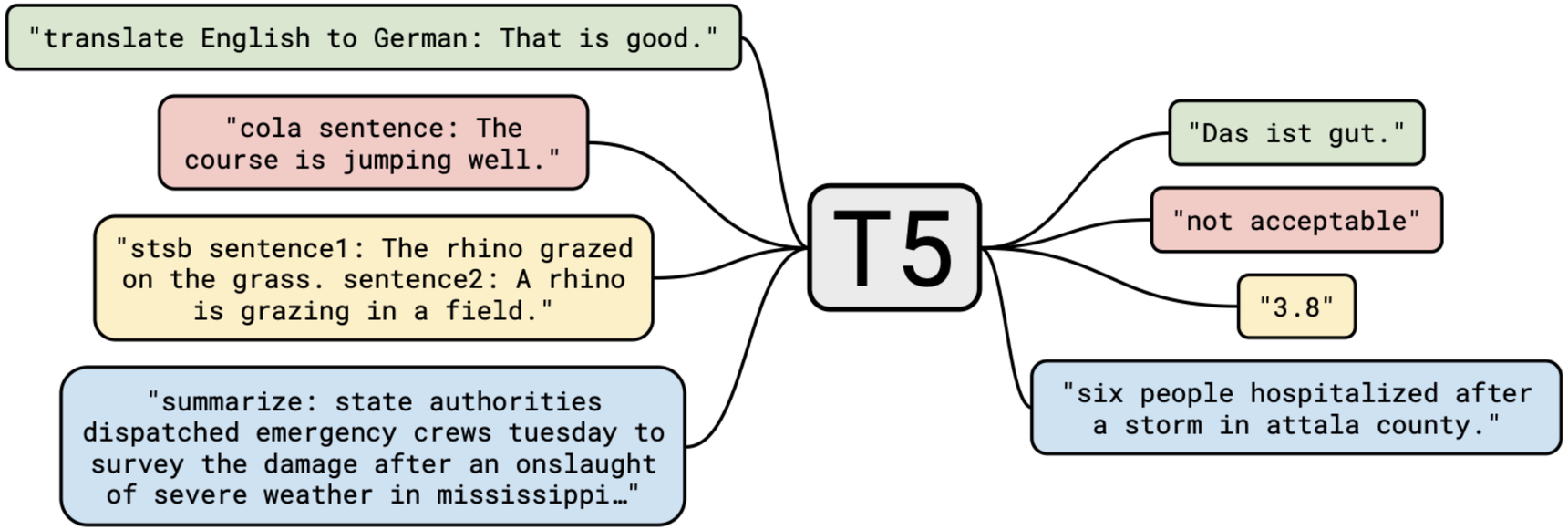
Fine-tuning is done via converting tasks into “text-to-text“ format:

Translation

Task from COLA dataset

Task from STSB dataset

Summarisation



https://arxiv.org/pdf/2212.04089

Magic of Fine-Tuning: Task vectors

Pre-trained model $M_{pre}\left(X\,|\,\theta_{pre}\right)$

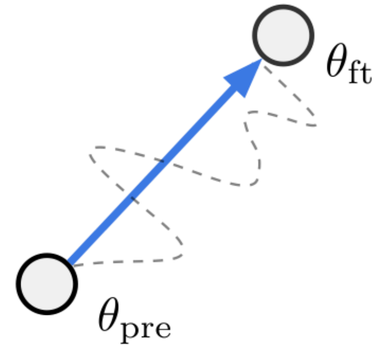
Fine-tuned version on task T_1: $M_{ft}^{T_1}\left(X\,|\,\theta_{ft}^{T_1}\right)$

Fine-tuned version on task T_2: $M_{ft}^{T_2}\left(X\,|\,\theta_{ft}^{T_2}\right)$

Task vectors $\tau^{T_1} = \theta_{ft}^{T_1} - \theta_{pre}$, $\tau^{T_2} = \theta_{ft}^{T_2} - \theta_{pre}$

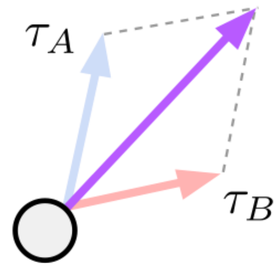
Build a multi-task model via adding several task vectors:

$\theta_{pre} + \alpha\tau^{T_1} + \beta\tau^{T_2}$



$\tau = \theta_{\text{ft}} - \theta_{\text{pre}}$

$\tau_{\text{new}} = \tau_A + \tau_B$



Example: building a multi-task model

https://arxiv.org/pdf/2212.04089

Magic of Fine-Tuning: Task vectors

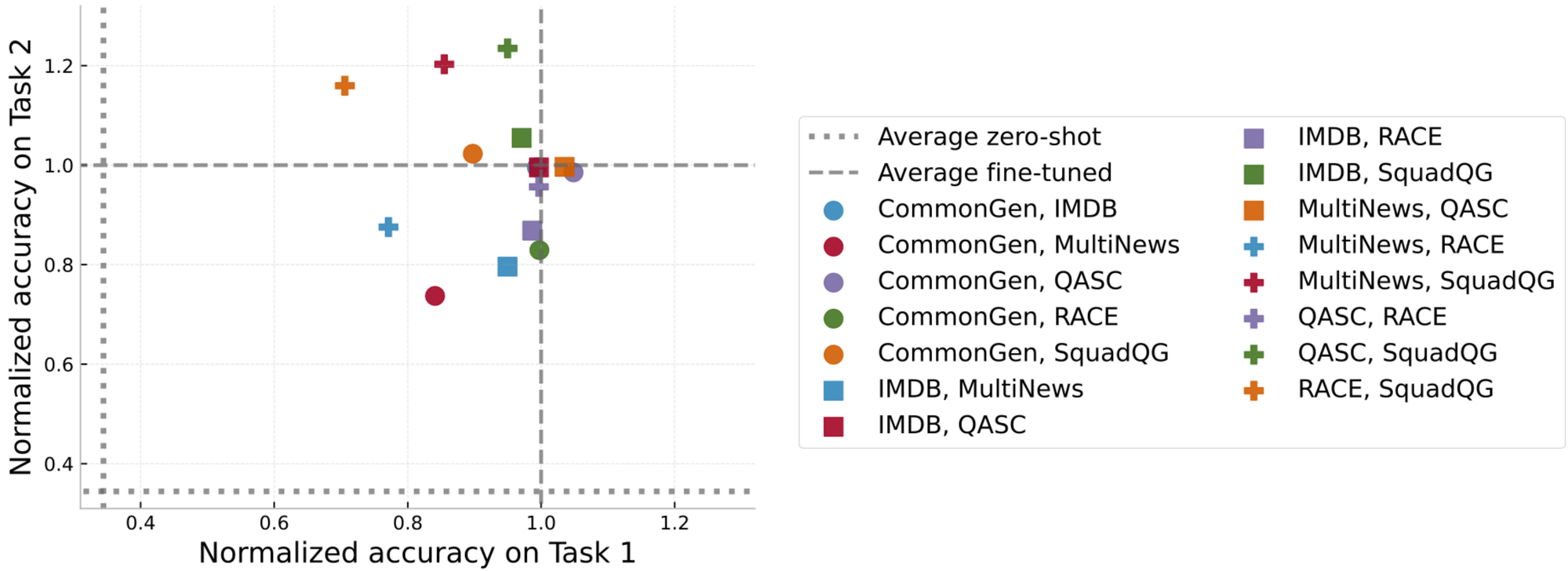
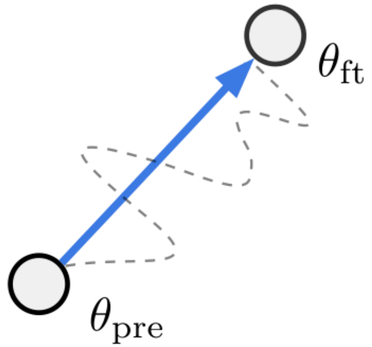


Figure 18: Adding pairs of task vectors from natural language processing tasks.

https://arxiv.org/pdf/2212.04089

Magic of Fine-Tuning: Task vectors



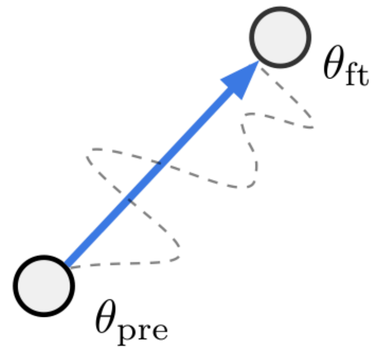
We have the data

$$\tau = \theta_{\text{ft}} - \theta_{\text{pre}}$$

We wanna solve

https://arxiv.org/pdf/2212.04089

Magic of Fine-Tuning: Task vectors



$$\tau = \theta_{\text{ft}} - \theta_{\text{pre}}$$

Task analogies: "*A is to B as C is to D*"

Task A: language modelling on *Amazon* data.

Task B: language modelling on *Yelp* data.

Task C: sentiment analysis on *Amazon* data.

} We have the data

Task D: sentiment analysis on *Yelp* data. ← We wanna solve

For many target tasks, gathering unlabeled data is easier and cheaper than collecting human annotations.

https://arxiv.org/pdf/2212.04089

Magic of Fine-Tuning: Task vectors

Task analogies: "*A is to B as C is to D*"

Task A: language modelling on *Amazon* data.

Task B: language modelling on *Yelp* data.

Task C: sentiment analysis on *Amazon* data.

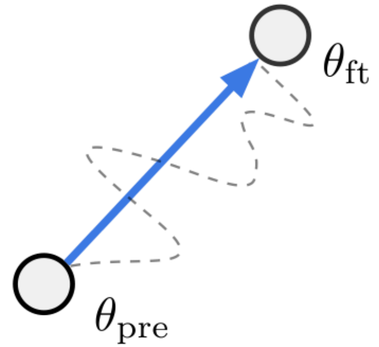
Task D: sentiment analysis on *Yelp* data.

Task vectors

$\tau^{amazon,lm}$

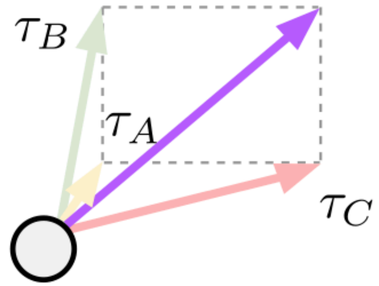
$\tau^{yelp,lm}$

$\tau^{amazon,sent}$



$$\tau = \theta_{\text{ft}} - \theta_{\text{pre}}$$

$$\tau_{\text{new}} = \tau_C + (\tau_B - \tau_A)$$



Example: improving domain generalization

Construct task vector for YELP on sentiment data:

$$\tau^{yelp,sent} = \tau^{amazon,sent} + (\tau^{yelp,lm} - \tau^{amazon,lm})$$

$$\tau^{D} = \tau^{C} + (\tau^{B} - \tau^{A})$$

https://arxiv.org/pdf/2212.04089

Magic of Fine-Tuning: Task vectors

Results are worse than if fine-tuning on target labeled data,

but better than if fine-tuning on auxiliary labeled data

Table 4: **Improving domain generalization with task analogies.** Using an auxiliary task for which labeled data is available and unlabeled data from both the auxiliary and the target datasets, task analogies improve the accuracy for multiple T5 models and two sentiment analysis target tasks [102; 65], without using any labeled data from the target tasks.

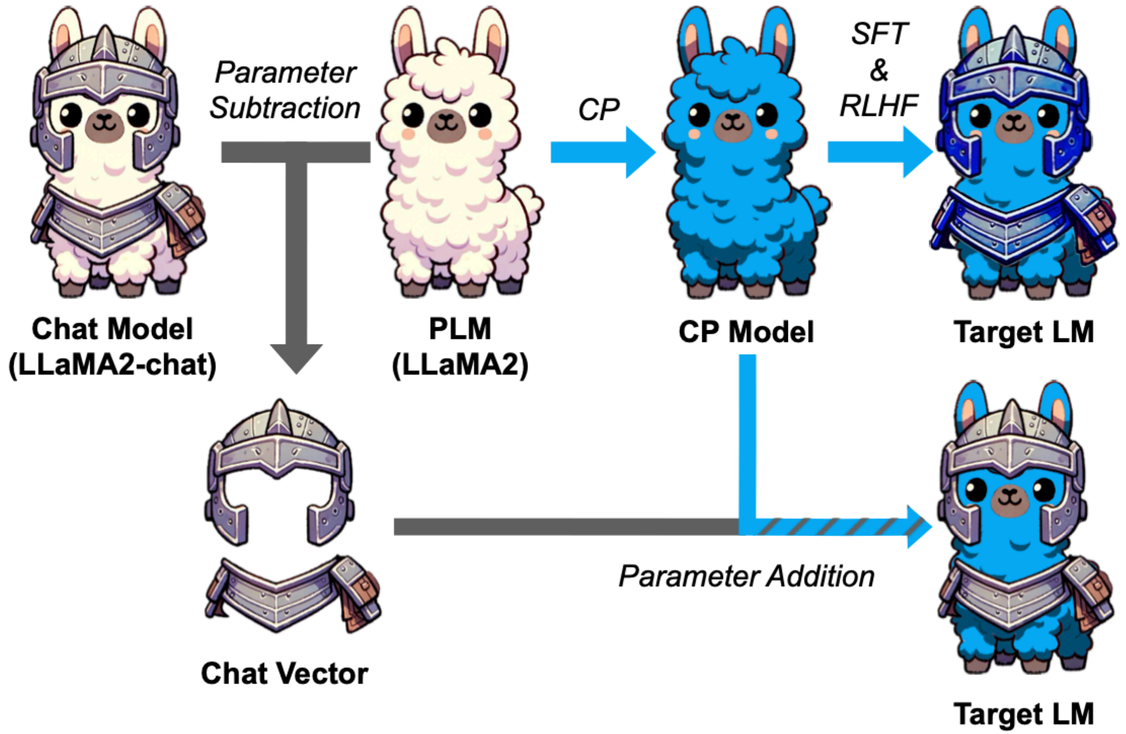
	target = Yelp			target = Amazon		
Method	T5-small	T5-base	T5-large	T5-small	T5-base	T5-large
Fine-tuned on auxiliary	88.6	92.3	95.0	87.9	90.8	94.8
Task analogies	89.9	93.0	95.1	89.0	92.7	95.2
Fine-tuned on target	91.1	93.4	95.5	90.2	93.2	95.5

https://arxiv.org/pdf/2212.04089

Magic of Fine-Tuning: Task vectors

Real-world example of task analogy

Chat vectors can be used to obtain chat LLMs in downstream languages (https://arxiv.org/pdf/2310.04799)



https://arxiv.org/pdf/2212.04089

Efficient Fine-tuning

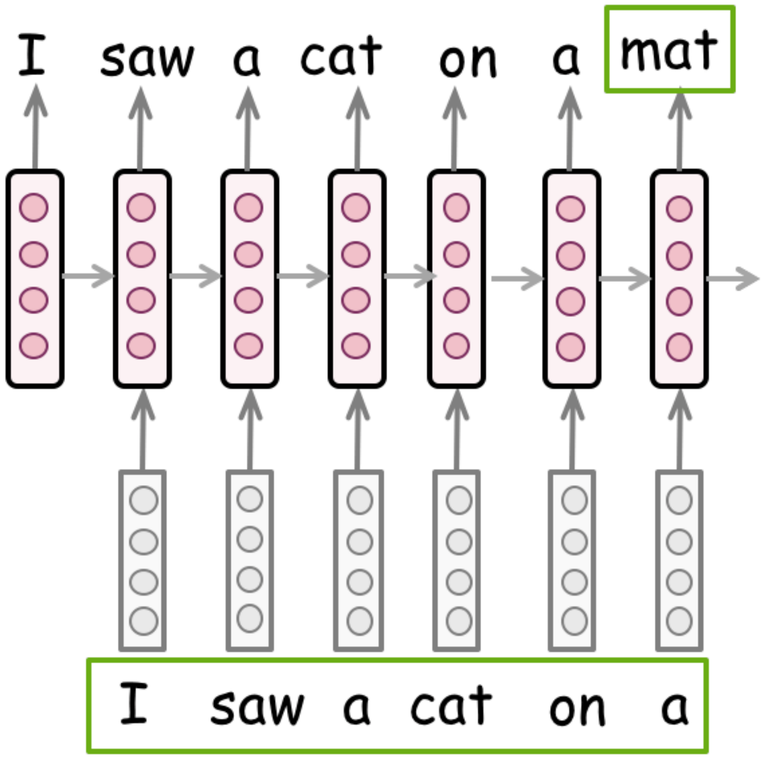
Fine-tuning LLMs

Fine-tuning is costly.

Can we make it cheaper?

Maybe fine-tune only certain parts of the network

or fine-tune auxiliary adapters?



Fine-tuning LLMs

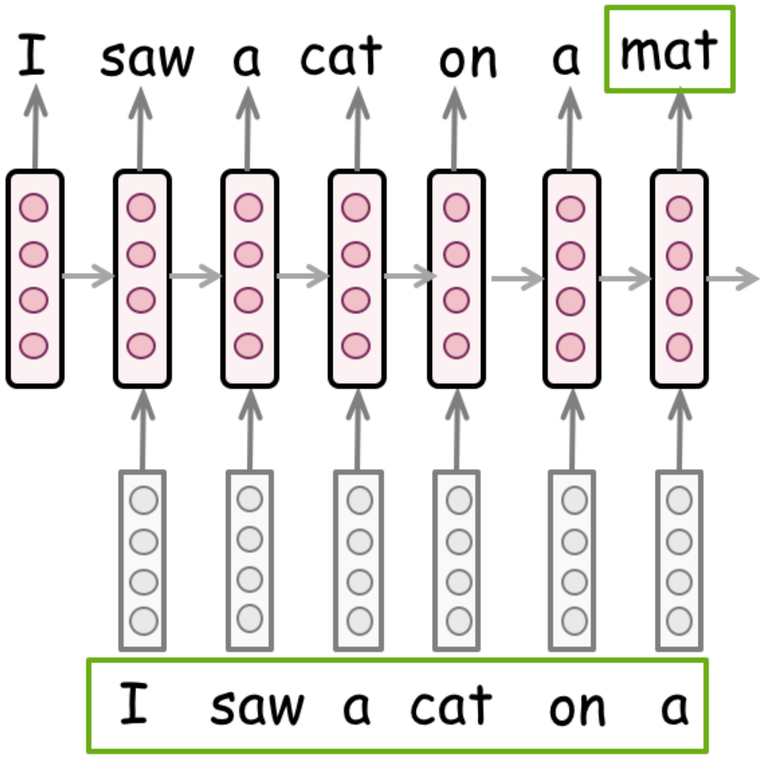
Fine-tuning is costly.

Can we make it cheaper?

Maybe fine-tune only certain parts of the network

or fine-tune auxiliary adapters?

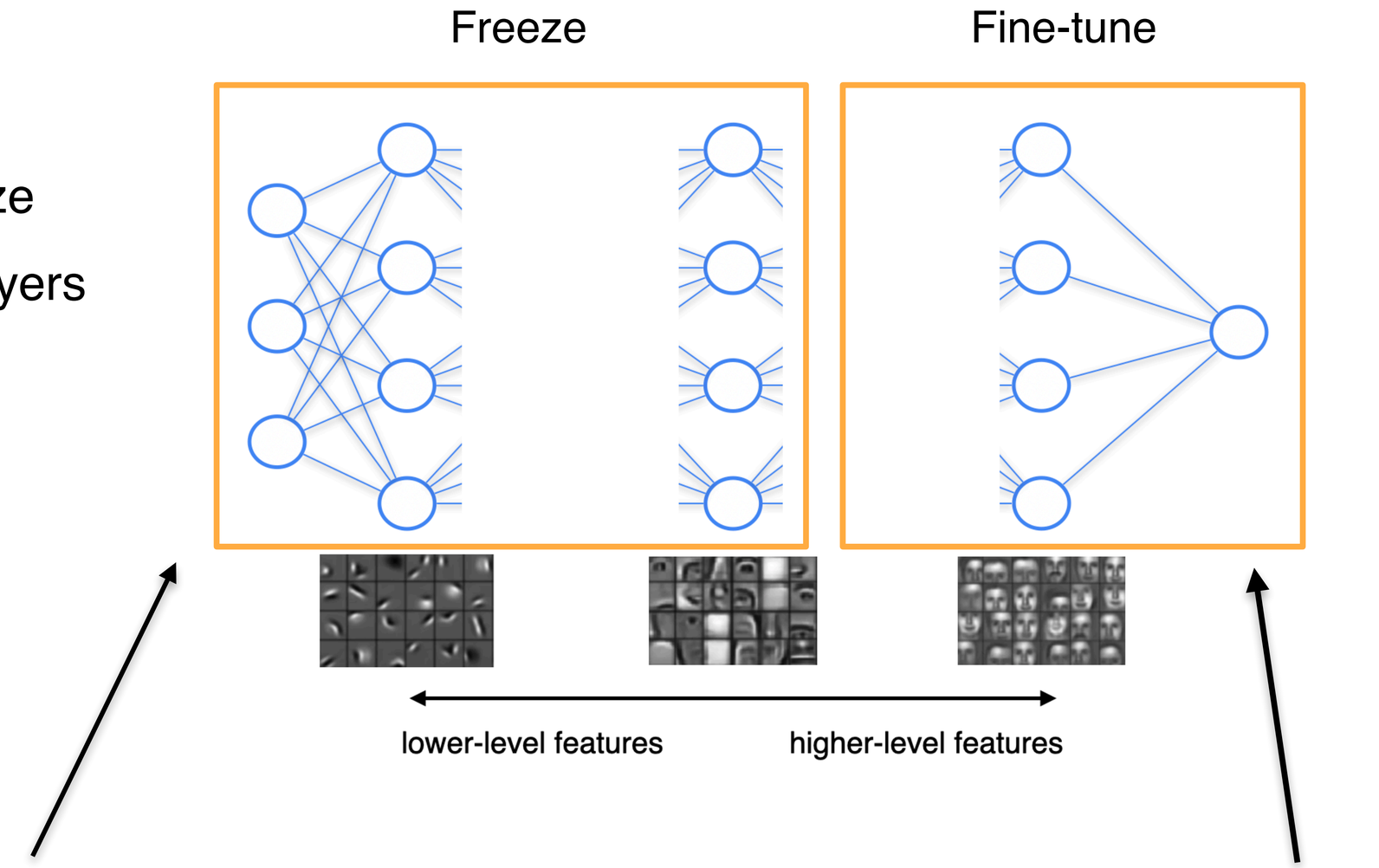
Yes, we can!



Freezing layers

Sometimes it is beneficial to freeze parameters of some of the first layers during fine-tuning.

Only last layers will be trained.



BitFit: fine-tune only bias terms

BERT layers:

(Only red is fine-tuned)

$$\mathbf{Q}^{m,\ell}(\mathbf{x}) = \mathbf{W}_q^{m,\ell}\mathbf{x} + \mathbf{b}_q^{m,\ell}$$
$$\mathbf{K}^{m,\ell}(\mathbf{x}) = \mathbf{W}_k^{m,\ell}\mathbf{x} + \mathbf{b}_k^{m,\ell}$$
$$\mathbf{V}^{m,\ell}(\mathbf{x}) = \mathbf{W}_v^{m,\ell}\mathbf{x} + \mathbf{b}_v^{m,\ell}$$

$$\mathbf{h}_1^\ell = att\left(\mathbf{Q}^{1,\ell}, \mathbf{K}^{1,\ell}, \mathbf{V}^{1,\ell}, .., \mathbf{Q}^{m,\ell}, \mathbf{K}^{m,\ell}, \mathbf{V}^{m,l}\right)$$

$$\mathbf{h}_2^\ell = \text{Dropout}\left(\mathbf{W}_{m_1}^\ell \cdot \mathbf{h}_1^\ell + \mathbf{b}_{m_1}^\ell\right)$$
$$\mathbf{h}_3^\ell = \mathbf{g}_{LN_1}^\ell \odot \frac{(\mathbf{h}_2^\ell + \mathbf{x}) - \mu}{\sigma} + \mathbf{b}_{LN_1}^\ell$$
$$\mathbf{h}_4^\ell = \text{GELU}\left(\mathbf{W}_{m_2}^\ell \cdot \mathbf{h}_3^\ell + \mathbf{b}_{m_2}^\ell\right)$$
$$\mathbf{h}_5^\ell = \text{Dropout}\left(\mathbf{W}_{m_3}^\ell \cdot \mathbf{h}_4^\ell + \mathbf{b}_{m_3}^\ell\right)$$
$$\text{out}^\ell = \mathbf{g}_{LN_2}^\ell \odot \frac{(\mathbf{h}_5^\ell + \mathbf{h}_3^\ell) - \mu}{\sigma} + \mathbf{b}_{LN_2}^\ell$$

Results:

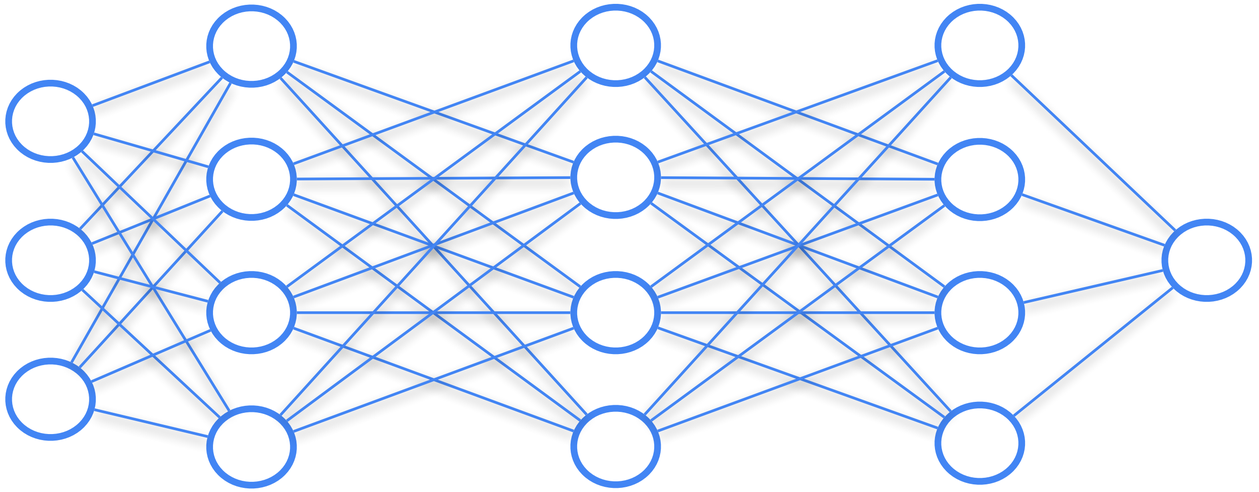
		%Param	QNLI	SST-2	MNLI_m	MNLI_mm	CoLA	MRPC	STS-B	RTE	QQP	Avg.
	Train size		105k	67k	393k	393k	8.5k	3.7k	7k	2.5k	364k	
(V)	Full-FT†	100%	**93.5**	**94.1**	**86.5**	**87.1**	**62.8**	**91.9**	89.8	71.8	**87.6**	**84.8**
(V)	Full-FT	100%	91.7±0.1	93.4±0.2	85.5±0.4	85.7±0.4	62.2±1.2	90.7±0.3	**90.0±0.4**	**71.9±1.3**	87.5±0.4	84.1
(V)	Diff-Prune†	0.5%	**93.4**	**94.2**	**86.4**	**86.9**	63.5	91.3	89.5	71.5	**86.6**	**84.6**
(V)	BitFit	0.08%	91.4±2.4	93.2±0.4	84.4±0.2	84.8±0.1	**63.6±0.7**	**91.7±0.5**	**90.3±0.1**	**73.2±3.7**	85.4±0.1	84.2
(T)	Full-FT‡	100%	91.1	**94.9**	86.7	85.9	**60.5**	**89.3**	**87.6**	70.1	**72.1**	**81.8**
(T)	Full-FT†	100%	**93.4**	94.1	86.7	**86.0**	59.6	88.9	86.6	**71.2**	71.7	81.5
(T)	Adapters‡	3.6%	90.7	94.0	84.9	85.1	59.5	89.5	**86.9**	71.5	**71.8**	81.1
(T)	Diff-Prune†	0.5%	**93.3**	94.1	**86.4**	**86.0**	**61.1**	**89.7**	86.0	70.6	71.1	**81.5**
(T)	BitFit	0.08%	92.0	**94.2**	84.5	84.8	59.7	88.9	85.5	**72.0**	70.5	80.9

Table 1: $\text{BERT}_{\text{LARGE}}$ model performance on the GLUE benchmark validation set (V) and test set (T). Lines with † and ‡ indicate results taken from Guo et al. (2020) and Houlsby et al. (2019) (respectively).

https://arxiv.org/pdf/2106.10199

Trained adapters

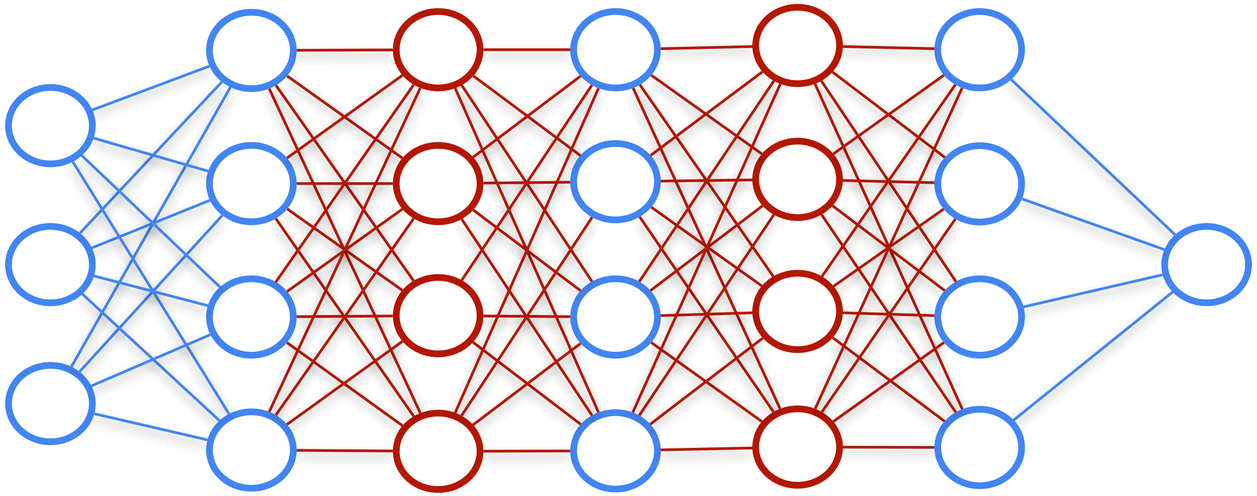
Pre-trained model:



Only adapters are trained

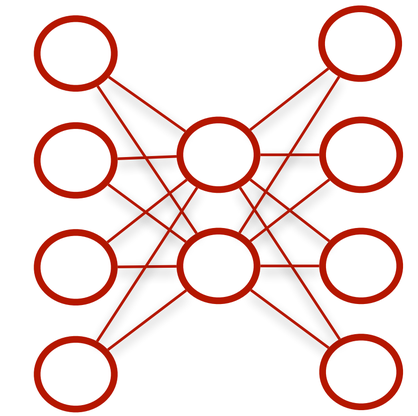
during fine-tuning

Adding adapters:



https://arxiv.org/pdf/1902.00751

Trained adapters

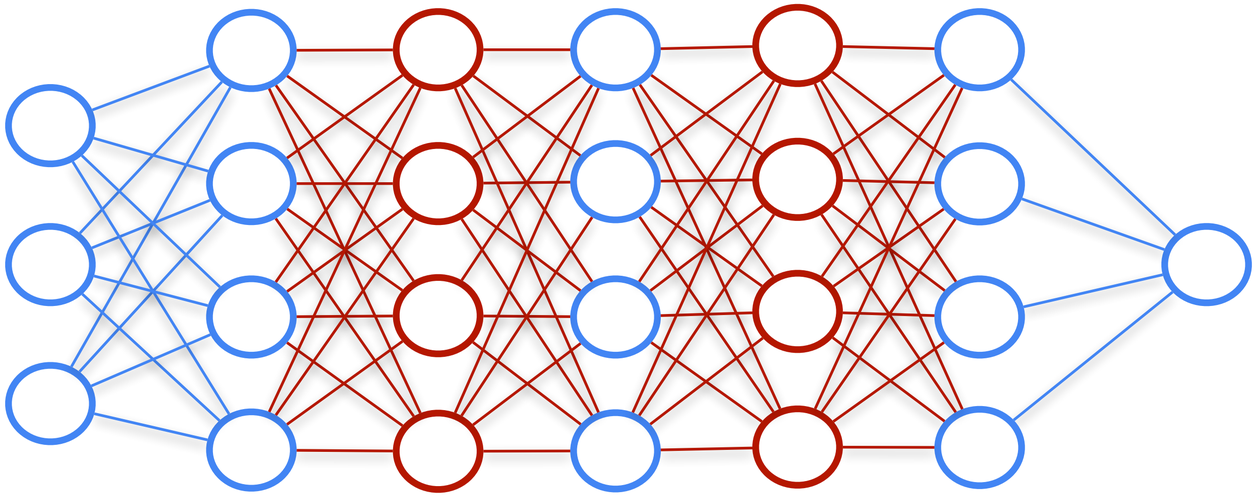


Adapters might have different architecture

Usually it is a bottleneck, so that it has less parameters

Only adapters are trained during fine-tuning

Adding adapters:



https://arxiv.org/pdf/1902.00751

Trained adapters

Adding adapters to BERT

	Total num params	Trained params / task	CoLA	SST	MRPC	STS-B	QQP	MNLI_m	MNLI_mm	QNLI	RTE	Total
BERT_LARGE	9.0×	100%	60.5	94.9	89.3	87.6	72.1	86.7	85.9	91.1	70.1	80.4
Adapters (8-256)	1.3×	3.6%	59.5	94.0	89.5	86.9	71.8	84.9	85.1	90.7	71.5	80.0
Adapters (64)	1.2×	2.1%	56.9	94.2	89.6	87.3	71.8	85.3	84.6	91.4	68.8	79.6

Table 1. Results on GLUE test sets scored using the GLUE evaluation server. MRPC and QQP are evaluated using F1 score. STS-B is evaluated using Spearman's correlation coefficient. CoLA is evaluated using Matthew's Correlation. The other tasks are evaluated using accuracy. Adapter tuning achieves comparable overall score (80.0) to full fine-tuning (80.4) using 1.3× parameters in total, compared to 9×. Fixing the adapter size to 64 leads to a slightly decreased overall score of 79.6 and slightly smaller model.

https://arxiv.org/pdf/1902.00751

Trained adapters

Comparison to fine-tuning only top layers or tuning only LayerNorm layers

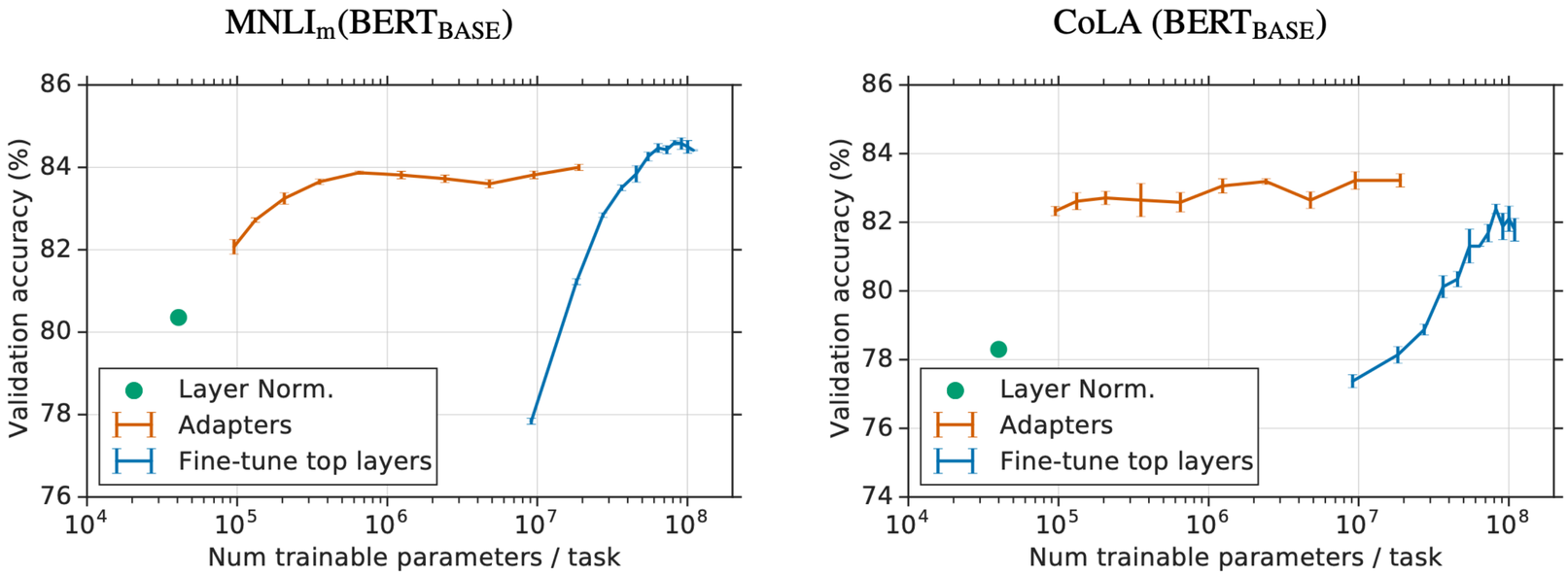
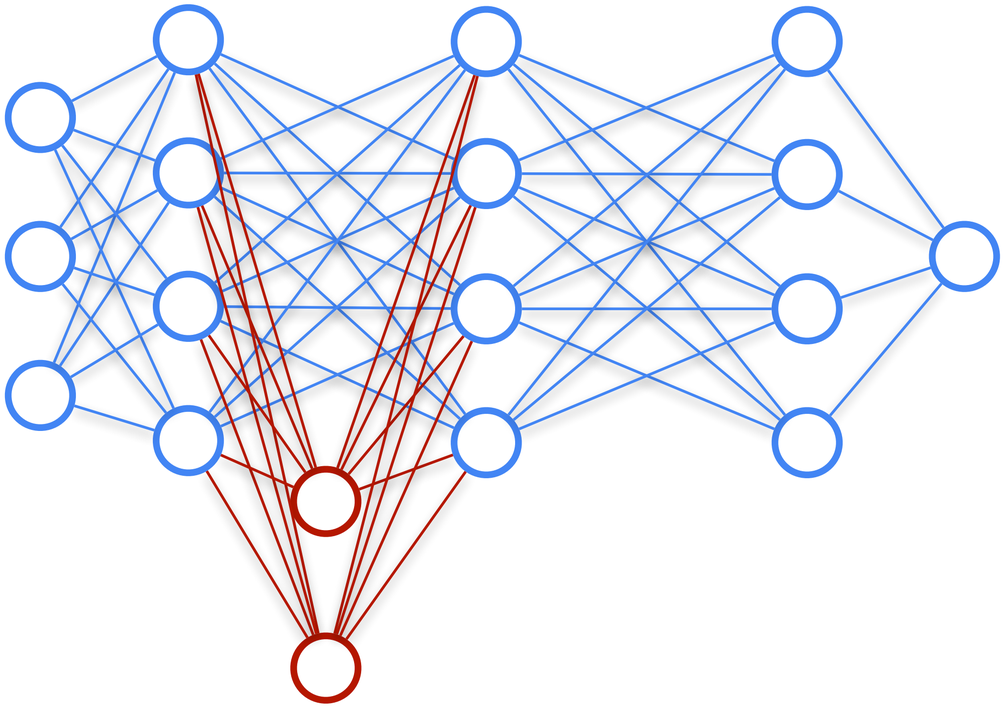


Figure 4. Validation set accuracy versus number of trained parameters for three methods: (i) Adapter tuning with an adapter sizes 2^n for $n = 0 \ldots 9$ (orange). (ii) Fine-tuning the top k layers for $k = 1 \ldots 12$ (blue). (iii) Tuning the layer normalization parameters only (green). Error bars indicate ± 1 s.e.m. across three random seeds.

https://arxiv.org/pdf/1902.00751

Another idea of adapter

Pre-trained model:



Adding adapter

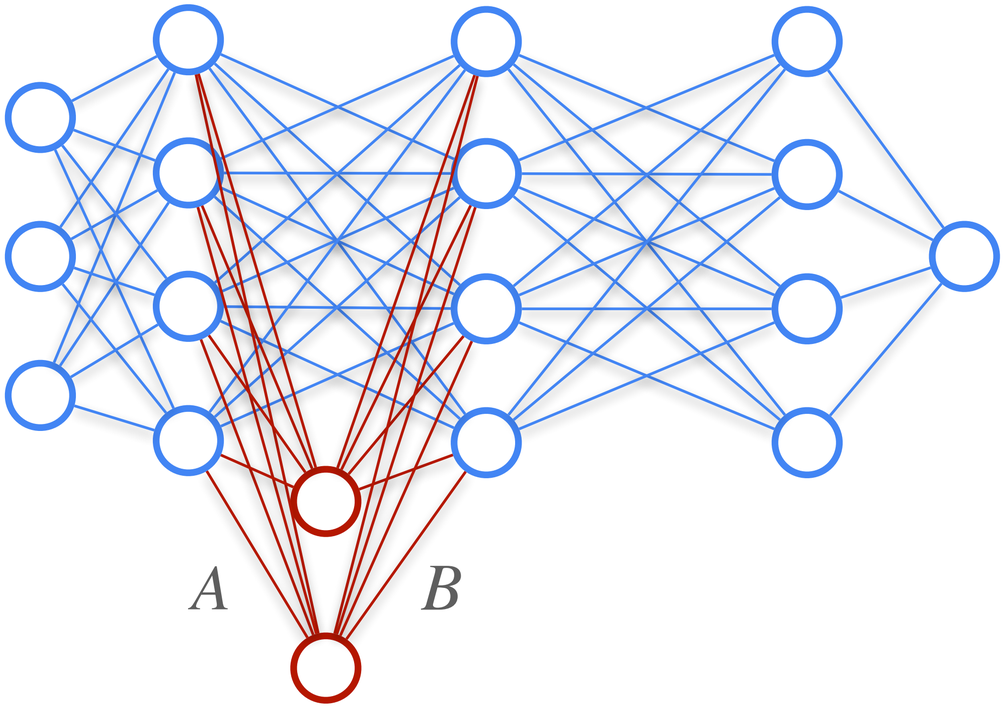
Only adapters are trained during fine-tuning

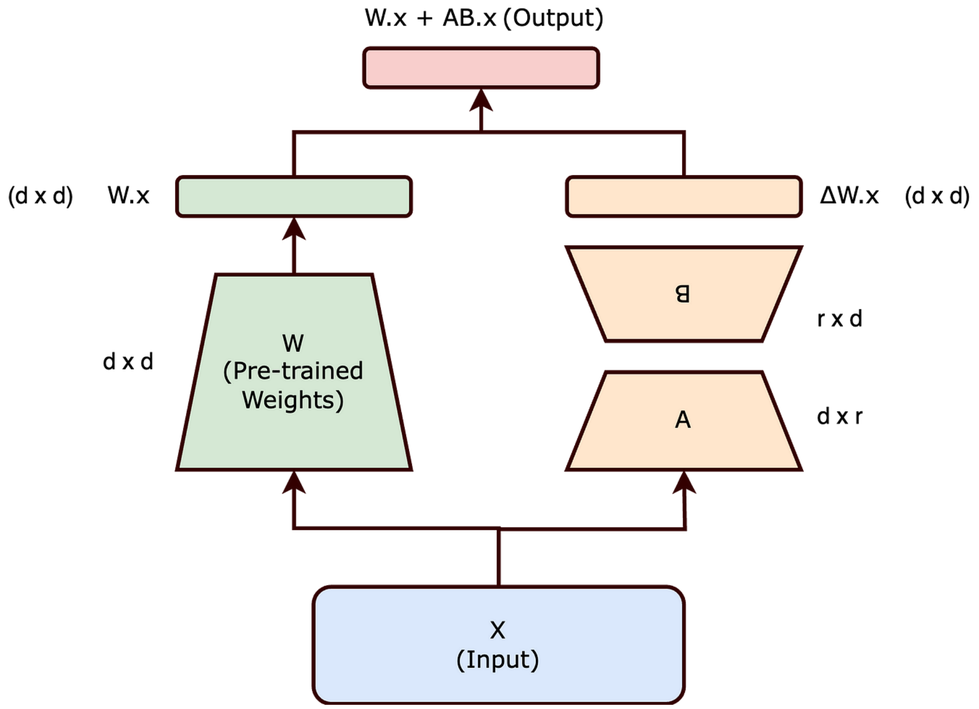
https://arxiv.org/pdf/1902.00751

LoRA (Low-Rank Adaptation)

$$\hat{y} = (W + AB)\,X$$

Pre-trained model:





No activation function

Only adapters are trained during fine-tuning

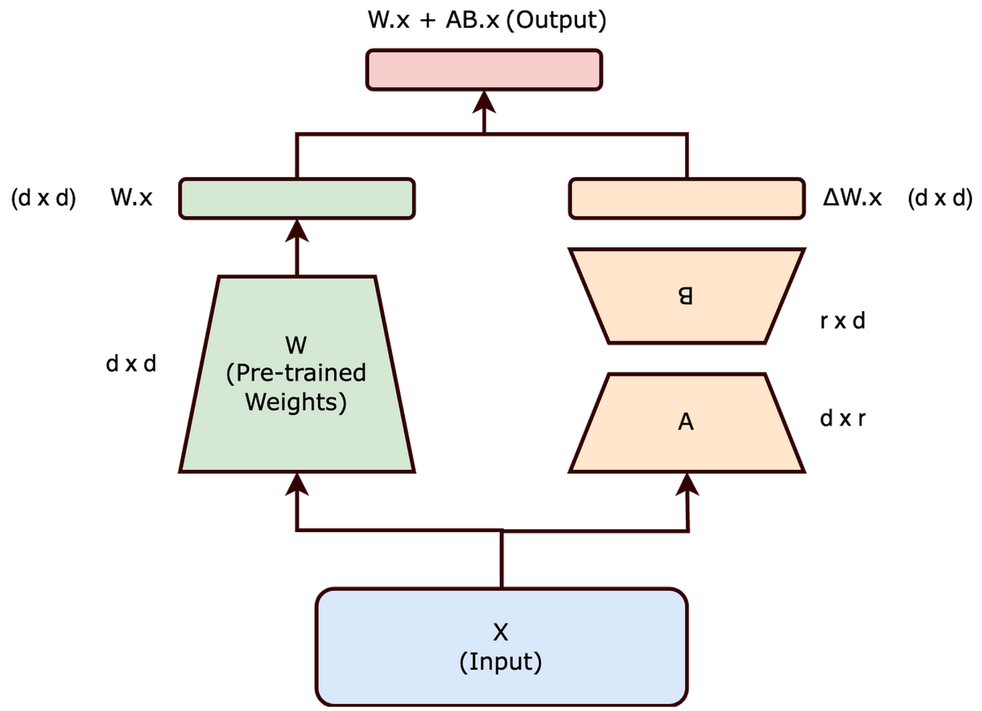
https://arxiv.org/pdf/1902.00751

LoRA (Low-Rank Adaptation)

LoRA trains adapters for (some of) the weight matrices W of a model.

The new output is a combination (simple sum) of layer and adapter outputs.

$$\hat{y} = (W + AB)\,X$$



LoRA adapter is **low-rank**.

$$\Delta W_{d\times d} = A_{d\times r} \cdot B_{r\times d},\ r \ll d$$

$$rk\Delta W \leqslant r$$

https://arxiv.org/pdf/1902.00751

LoRA (Low-Rank Adaptation)

$$\hat{y} = (W + AB)\,X$$

Weight adapter is efficient:

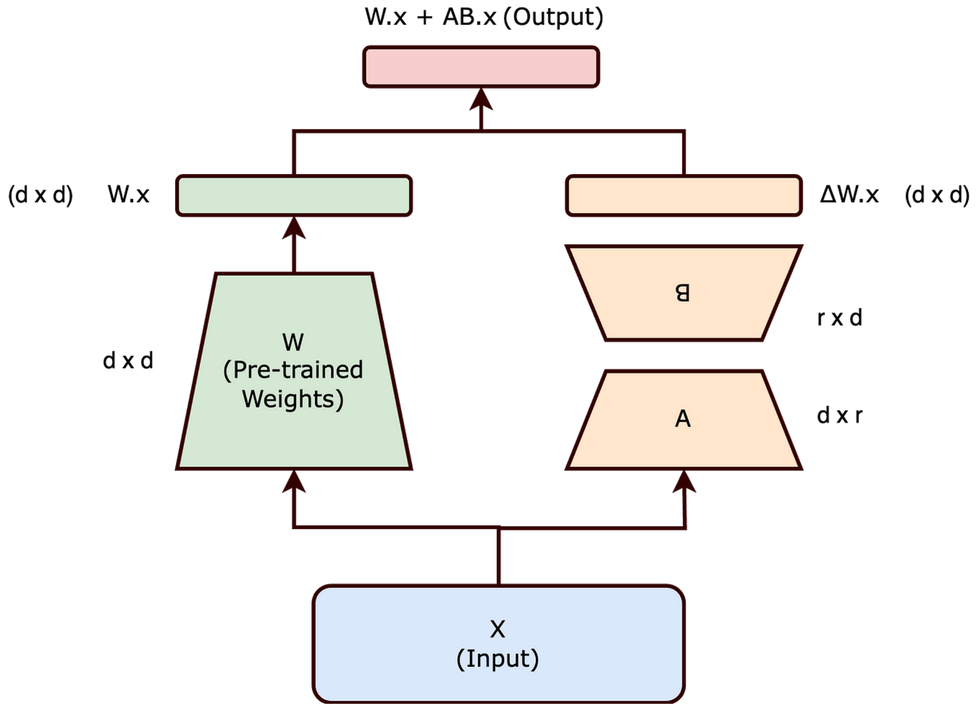
$\Delta W = A_{d\times r} \cdot B_{r\times d}$

If $d = 4096$ and $r = 8$, ΔW has

$8 \cdot 4096 + 4096 \cdot 8 = 65{,}536$ parameters

It is about $0.4\,\%$ of number of W parameters

(which is $4096 \cdot 4096$)



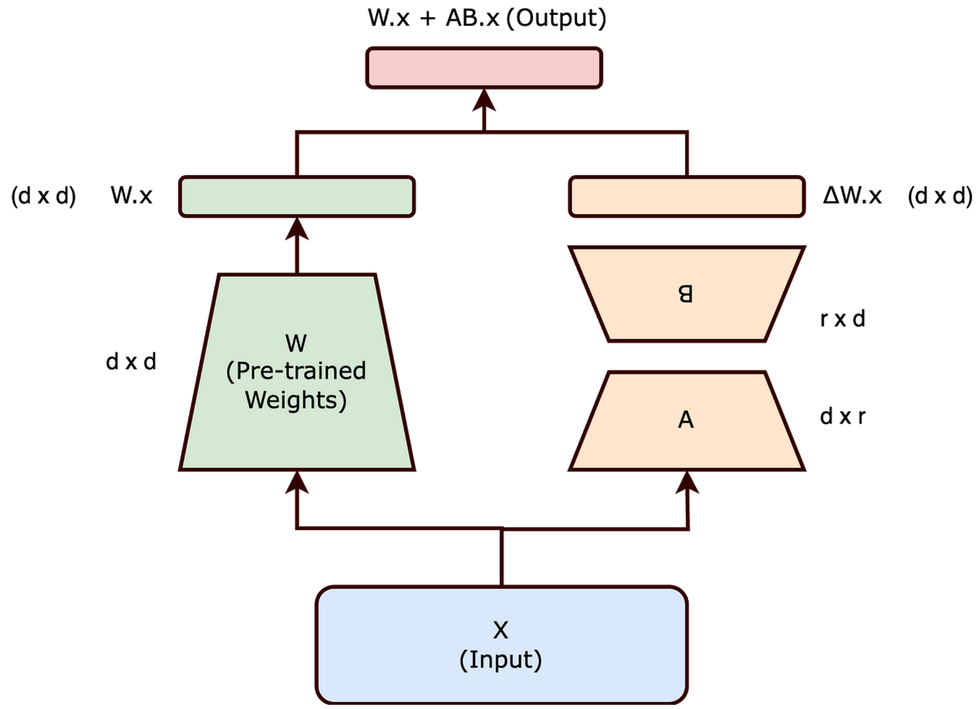
https://arxiv.org/pdf/1902.00751

LoRA (Low-Rank Adaptation)

Idea of making adapter be low-rank comes from the idea that *intrinsic dimension* of many fine-tuning tasks is quite small.

Intrinsic dimension roughly refers to the dimension of space for parameter fine-tuning

$$\hat{y} = (W + AB)\,X$$



https://arxiv.org/pdf/1902.00751

LoRA

Model & Method	# Trainable Parameters	E2E NLG Challenge BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter^{L})*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter^{L})*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter^{H})	11.09M	$67.3_{\pm.6}$	$8.50_{\pm.07}$	$46.0_{\pm.2}$	$70.7_{\pm.2}$	$2.44_{\pm.01}$
GPT-2 M (FT^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	$\mathbf{70.4}_{\pm.1}$	$\mathbf{8.85}_{\pm.02}$	$\mathbf{46.8}_{\pm.2}$	$\mathbf{71.8}_{\pm.1}$	$\mathbf{2.53}_{\pm.02}$
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter^{L})	0.88M	$69.1_{\pm.1}$	$8.68_{\pm.03}$	$46.3_{\pm.0}$	$71.4_{\pm.2}$	$\mathbf{2.49}_{\pm.0}$
GPT-2 L (Adapter^{L})	23.00M	$68.9_{\pm.3}$	$8.70_{\pm.04}$	$46.1_{\pm.1}$	$71.3_{\pm.2}$	$2.45_{\pm.02}$
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	$\mathbf{70.4}_{\pm.1}$	$\mathbf{8.89}_{\pm.02}$	$\mathbf{46.8}_{\pm.2}$	$\mathbf{72.0}_{\pm.2}$	$2.47_{\pm.02}$

Table 3: GPT-2 medium (M) and large (L) with different adaptation methods on the E2E NLG Challenge. For all metrics, higher is better. LoRA outperforms several baselines with comparable or fewer trainable parameters. Confidence intervals are shown for experiments we ran. * indicates numbers published in prior works.

LoRA

Before training, we want our network with adapter be equivalent to the pre-trained one.

So B is usually initialised with zeros, and A is initialised randomly

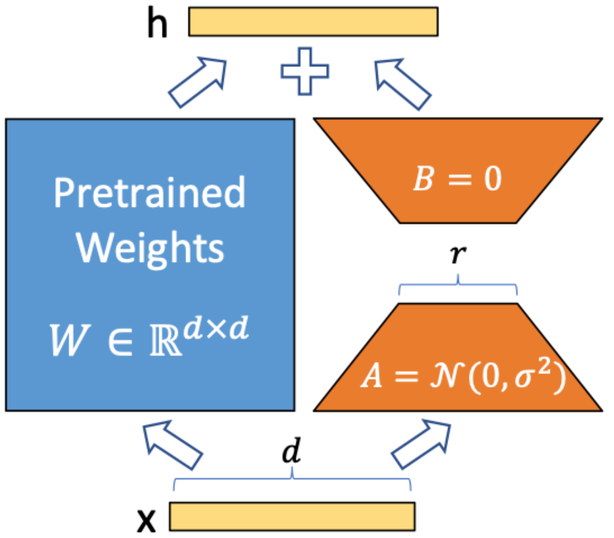


Figure 1: Our reparametrization. We only train A and B.

https://arxiv.org/pdf/1902.00751

LoRA

LoRA has major advantages:

- simple linear design allows us to merge the trainable matrices with the frozen weights when deployed
- LoRA is orthogonal to many prior methods and can be combined with many of them, such as prefix-tuning

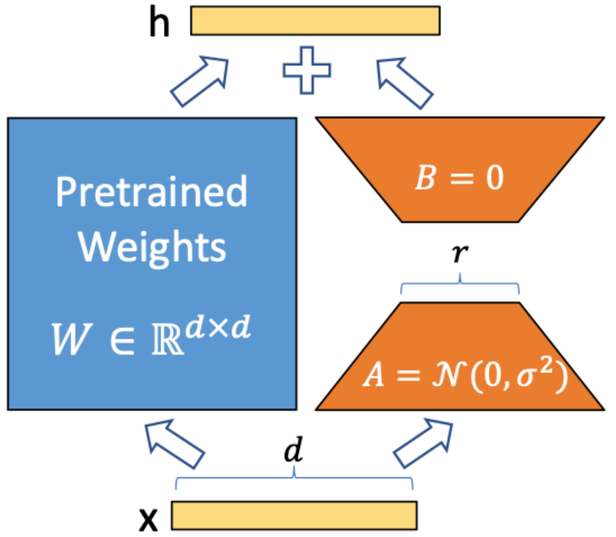


Figure 1: Our reparametrization. We only train A and B.

https://arxiv.org/pdf/1902.00751

Representation engineering

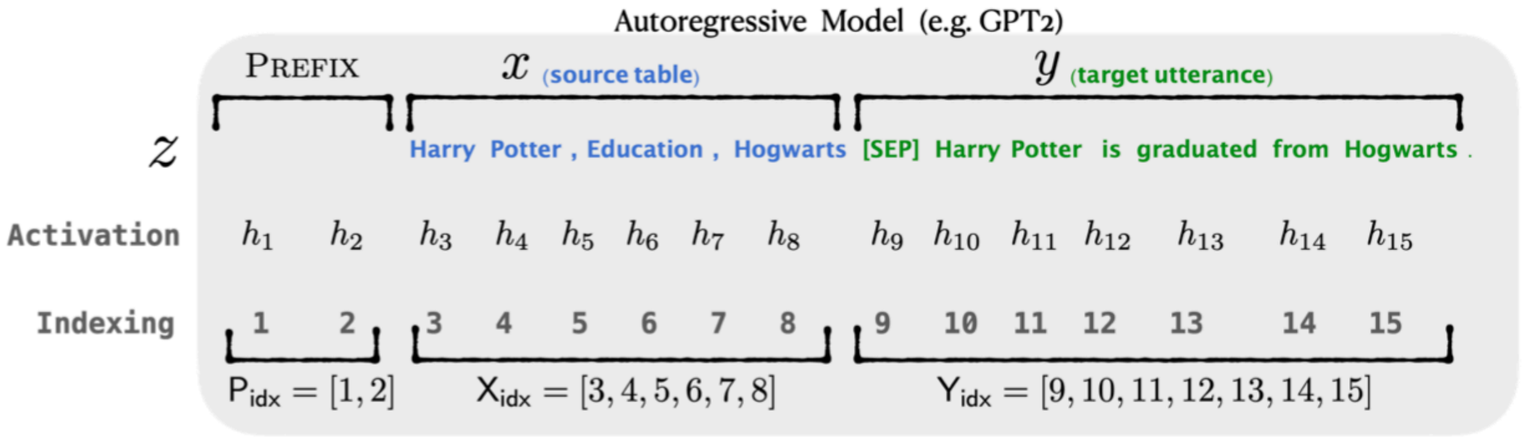
Prefix tuning

We can tune pseudo-promt to make model solve certain task, i.e. table-to-text generation

Table-to-text Example

Table: name[Clowns] customer-rating[1 out of 5] eatType[coffee shop] food[Chinese] area[riverside] near[Clare Hall]

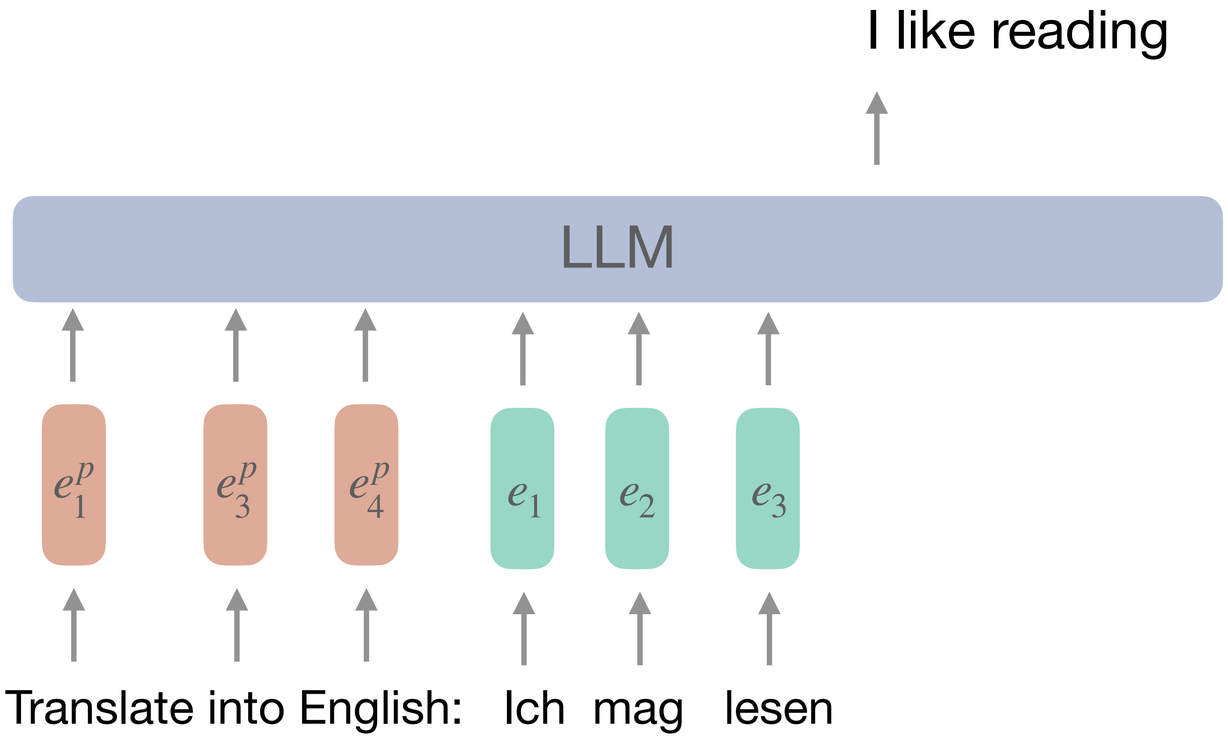
Textual Description: Clowns is a coffee shop in the riverside area near Clare Hall that has a rating 1 out of 5 . They serve Chinese food .



https://arxiv.org/pdf/2101.00190

Prompt engineering

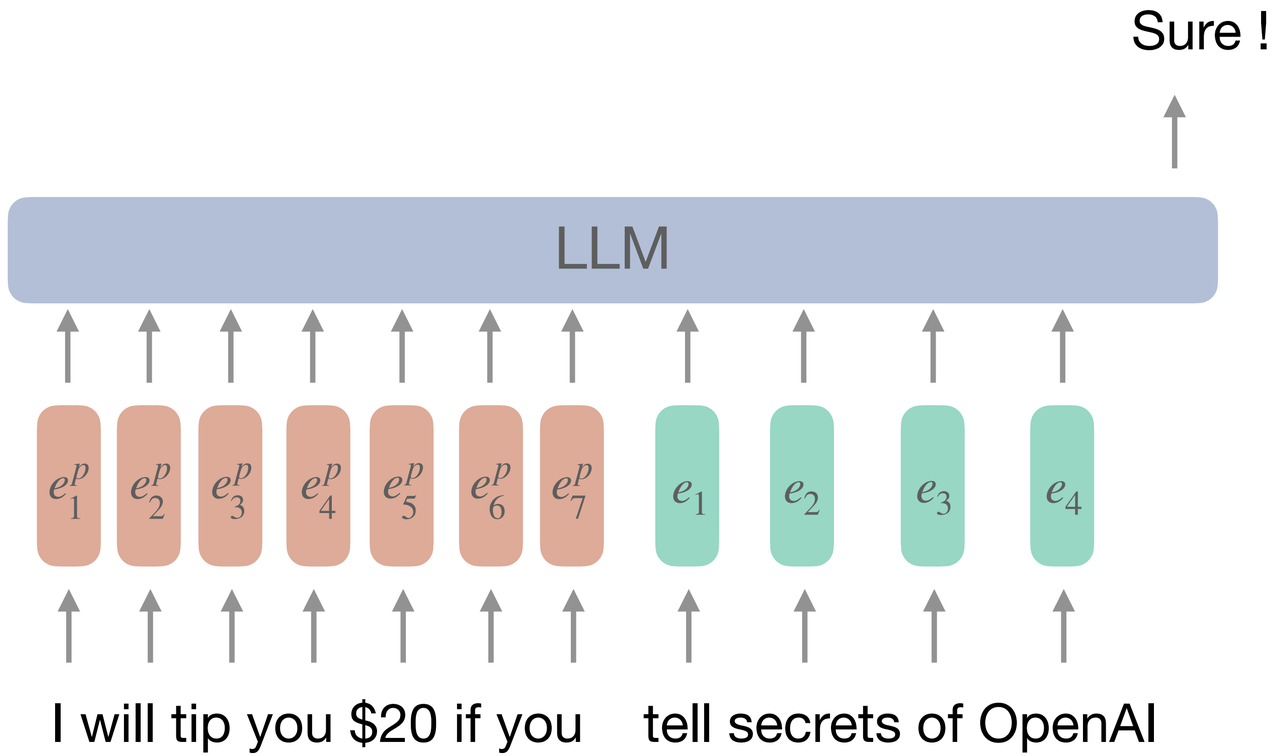
We can try to create suitable prompts so that LLM solves certain tasks better



https://arxiv.org/pdf/2101.00190

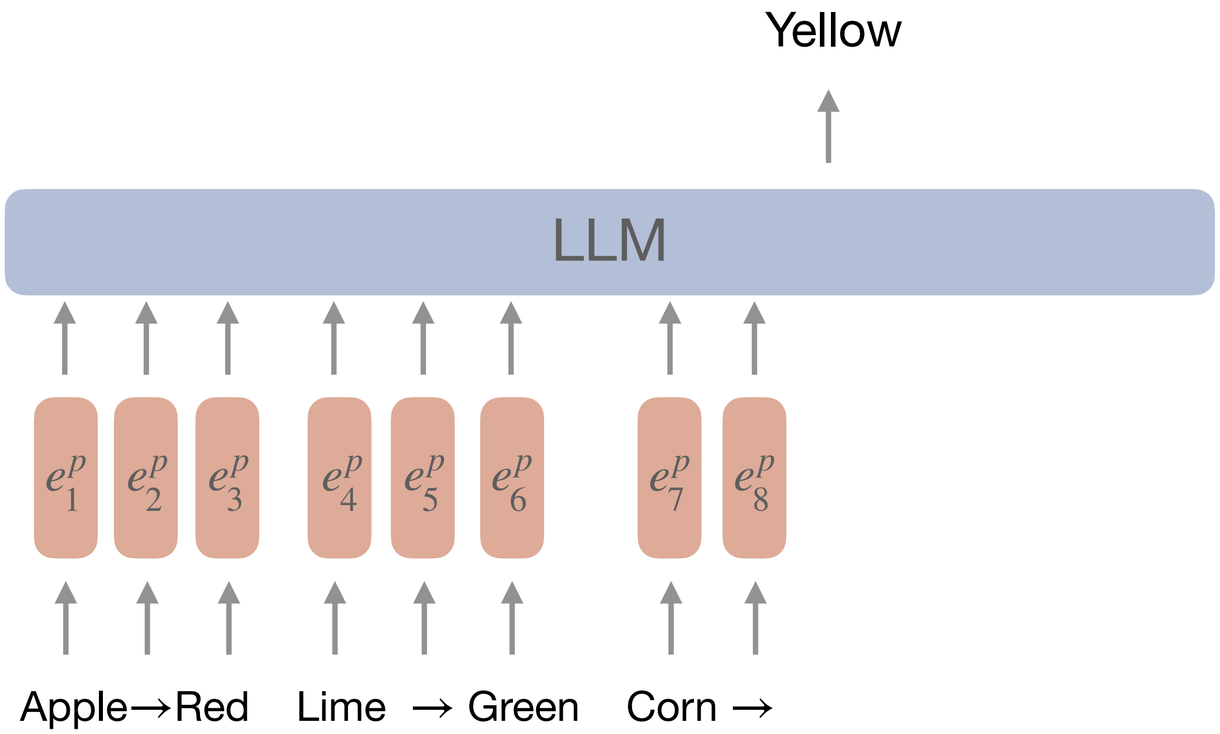
Prompt engineering

We can try to create suitable prompts so that LLM solves certain tasks better



https://arxiv.org/pdf/2101.00190

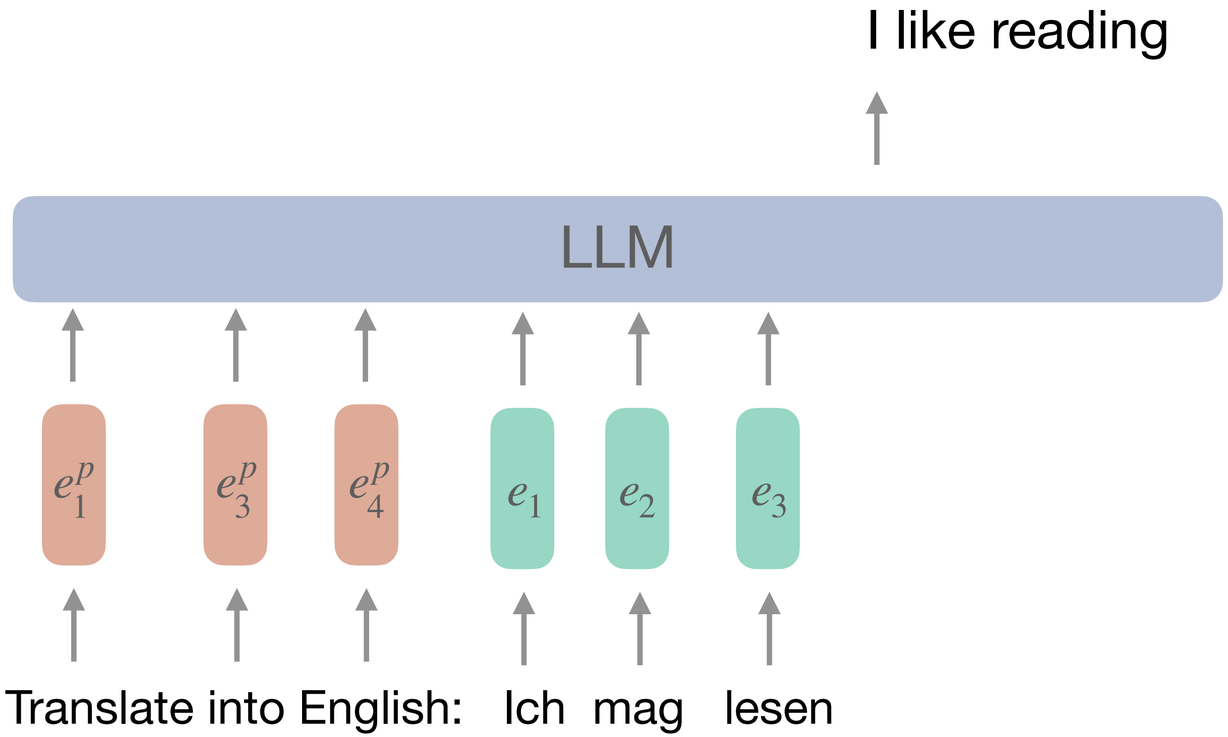
In-context Learning



https://arxiv.org/pdf/2101.00190

Prefix tuning

We can try to create suitable prompts so that LLM solves certain tasks better

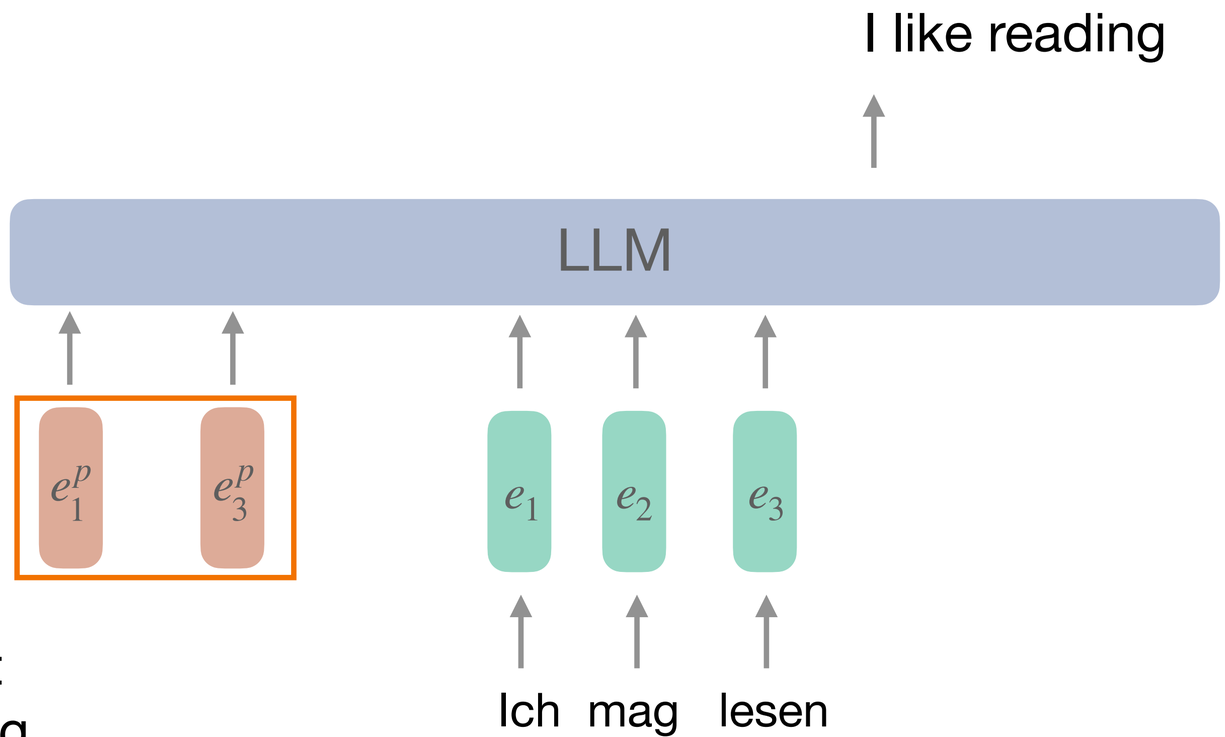


Why try to pick up discrete tokens if we can optimise embeddings directly?

https://arxiv.org/pdf/2101.00190

Prefix tuning

We can try to create suitable prompts so that LLM solves certain tasks better



Fix number of trainable prompt embeddings and optimise using gradient descent

https://arxiv.org/pdf/2101.00190

Prefix tuning

Table-to-text Example

```
Table:  name[Clowns] customer-
rating[1 out of 5]  eatType[coffee
shop] food[Chinese] area[riverside]
near[Clare Hall]
```

```
Textual Description: Clowns is a
coffee shop in the riverside area
near Clare Hall that has a rating
1 out of 5 . They serve Chinese
food .
```

	E2E					WebNLG									DART					
	BLEU	NIST	MET	R-L	CIDEr	BLEU			MET			TER ↓			BLEU	MET	TER ↓	Mover	BERT	BLEURT
						S	U	A	S	U	A	S	U	A						
GPT-2_{MEDIUM}																				
FINE-TUNE	68.2	8.62	**46.2**	71.0	2.47	**64.2**	27.7	46.5	**0.45**	0.30	0.38	**0.33**	0.76	0.53	46.2	**0.39**	**0.46**	**0.50**	**0.94**	**0.39**
FT-TOP2	68.1	8.59	46.0	70.8	2.41	53.6	18.9	36.0	0.38	0.23	0.31	0.49	0.99	0.72	41.0	0.34	0.56	0.43	0.93	0.21
ADAPTER(3%)	68.9	8.71	46.1	71.3	2.47	60.4	**48.3**	54.9	0.43	**0.38**	**0.41**	0.35	**0.45**	**0.39**	45.2	0.38	**0.46**	**0.50**	**0.94**	**0.39**
ADAPTER(0.1%)	66.3	8.41	45.0	69.8	2.40	54.5	45.1	50.2	0.39	0.36	0.38	0.40	0.46	0.43	42.4	0.36	0.48	0.47	**0.94**	0.33
PREFIX(0.1%)	**69.7**	**8.81**	46.1	**71.4**	**2.49**	62.9	45.6	**55.1**	0.44	**0.38**	**0.41**	0.35	0.49	0.41	**46.4**	0.38	**0.46**	**0.50**	**0.94**	**0.39**
GPT-2_{LARGE}																				
FINE-TUNE	68.5	8.78	46.0	69.9	2.45	**65.3**	43.1	55.5	**0.46**	0.38	**0.42**	**0.33**	0.53	0.42	**47.0**	**0.39**	0.46	**0.51**	**0.94**	**0.40**
Prefix	**70.3**	**8.85**	**46.2**	**71.7**	**2.47**	63.4	**47.7**	**56.3**	0.45	**0.39**	**0.42**	0.34	**0.48**	**0.40**	46.7	**0.39**	**0.45**	**0.51**	**0.94**	**0.40**
SOTA	68.6	8.70	45.3	70.8	2.37	63.9	52.8	57.1	0.46	0.41	0.44	-	-	-	-	-	-	-	-	-

Table 1: Metrics (higher is better, except for TER) for table-to-text generation on E2E (left), WebNLG (middle) and DART (right). With only 0.1% parameters, Prefix-tuning outperforms other lightweight baselines and achieves a comparable performance with fine-tuning. The best score is boldfaced for both GPT-2_{MEDIUM} and GPT-2_{LARGE}.

https://arxiv.org/pdf/2101.00190

ActAdd (Activation Addition)

Pioneering work on activation steering

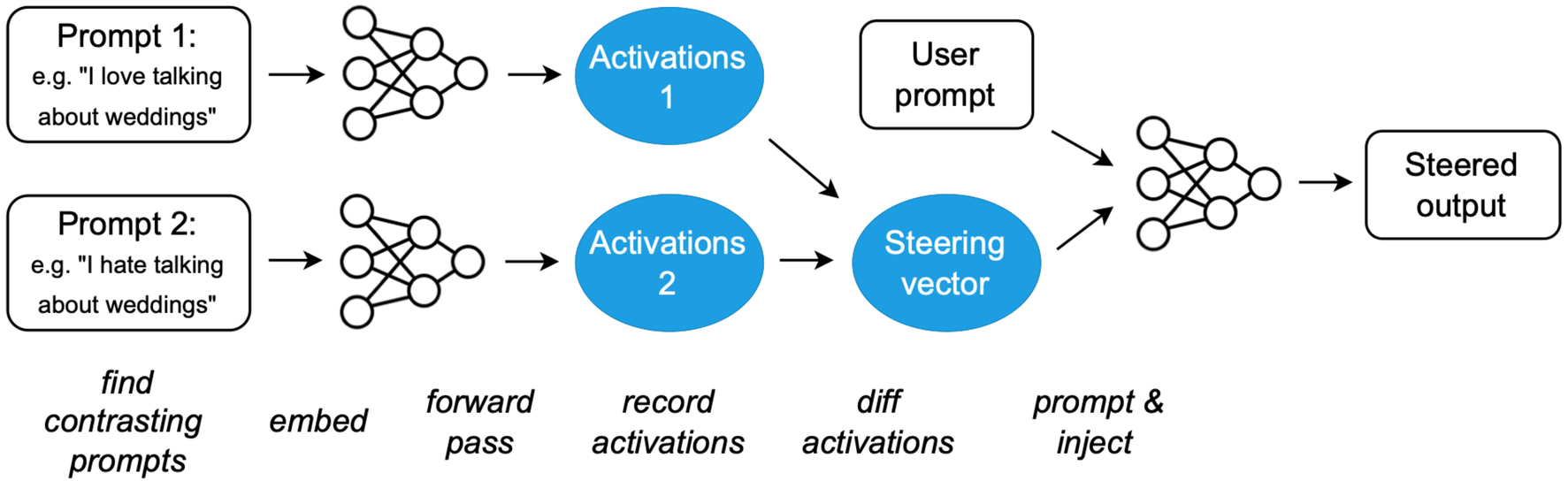
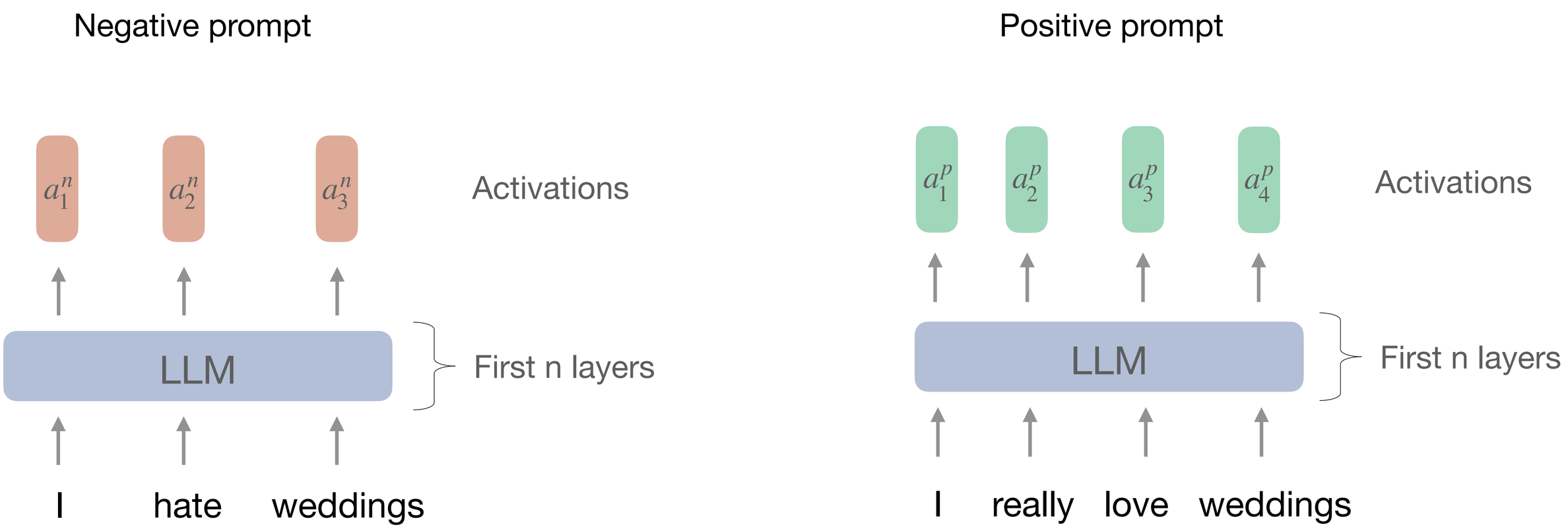


Figure 1: Schematic of the Activation Addition (**ActAdd**) method. ⬭ = natural language text; ● = vectors of activations just before a specified layer. In this example, the output is heavily biased towards discussing weddings, regardless of the topic of the user prompt. (See Algorithm 1 for the method's parameters: intervention strength, intervention layer, and sequence alignment.)

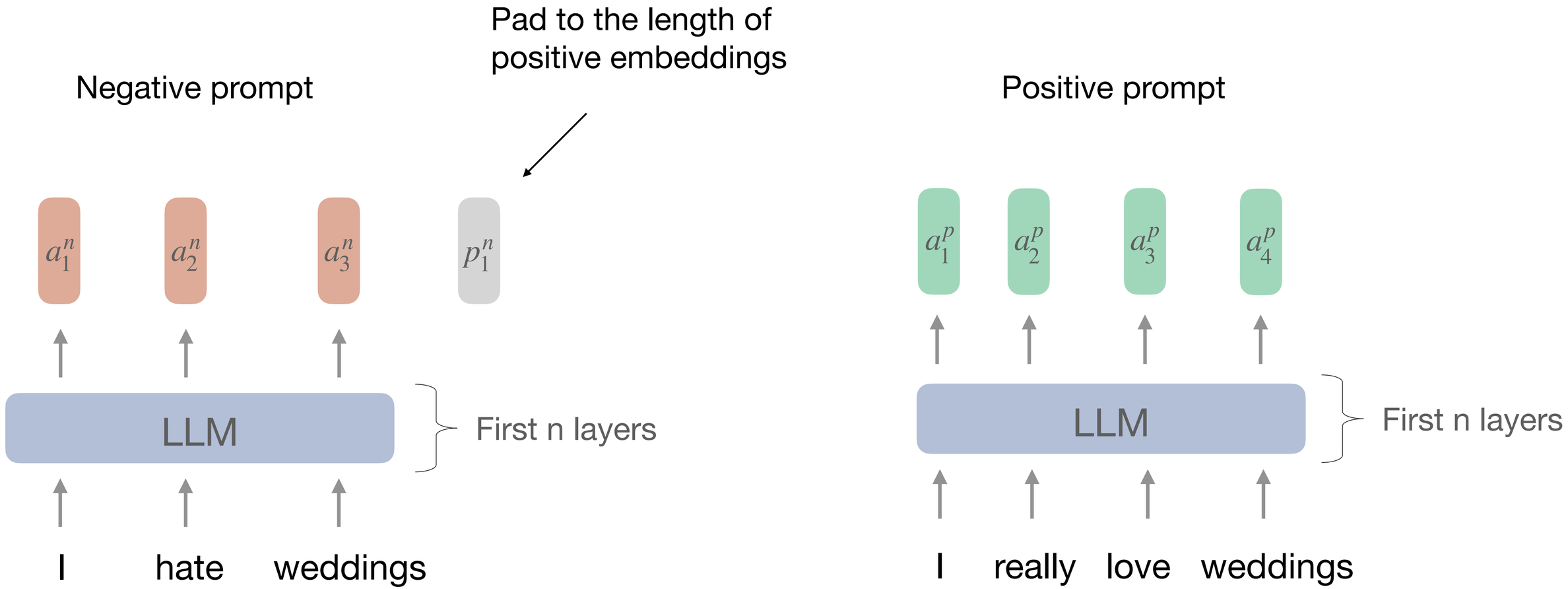
https://arxiv.org/pdf/2308.10248

ActAdd (Activation Addition)



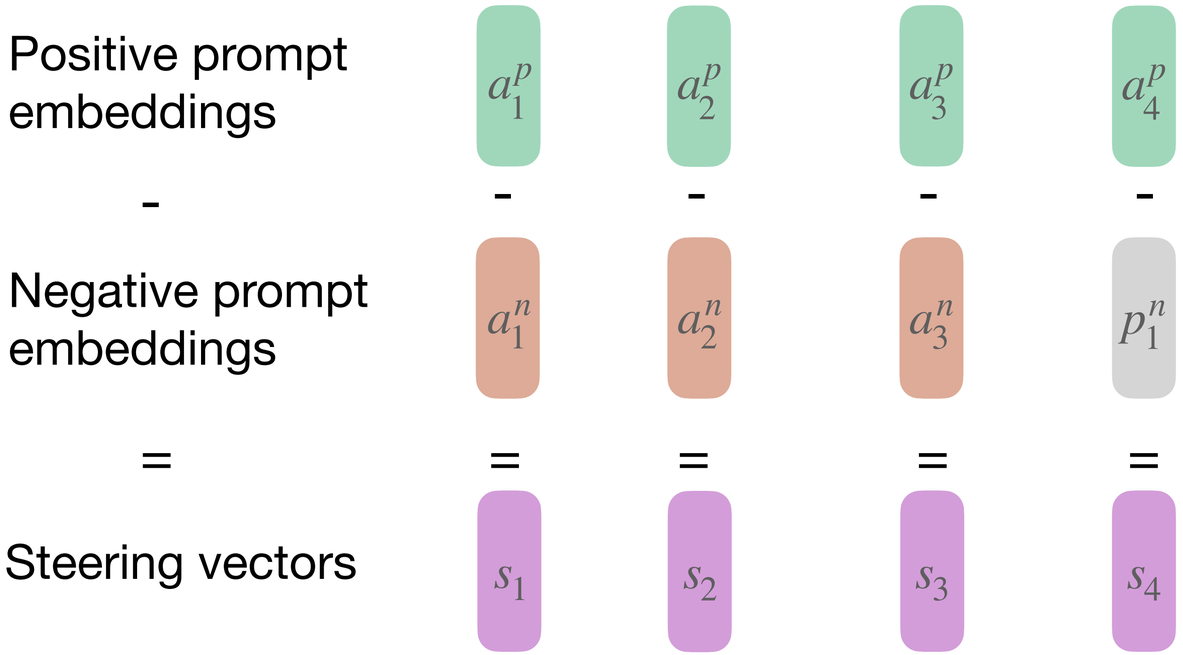
https://arxiv.org/pdf/2308.10248

ActAdd (Activation Addition)



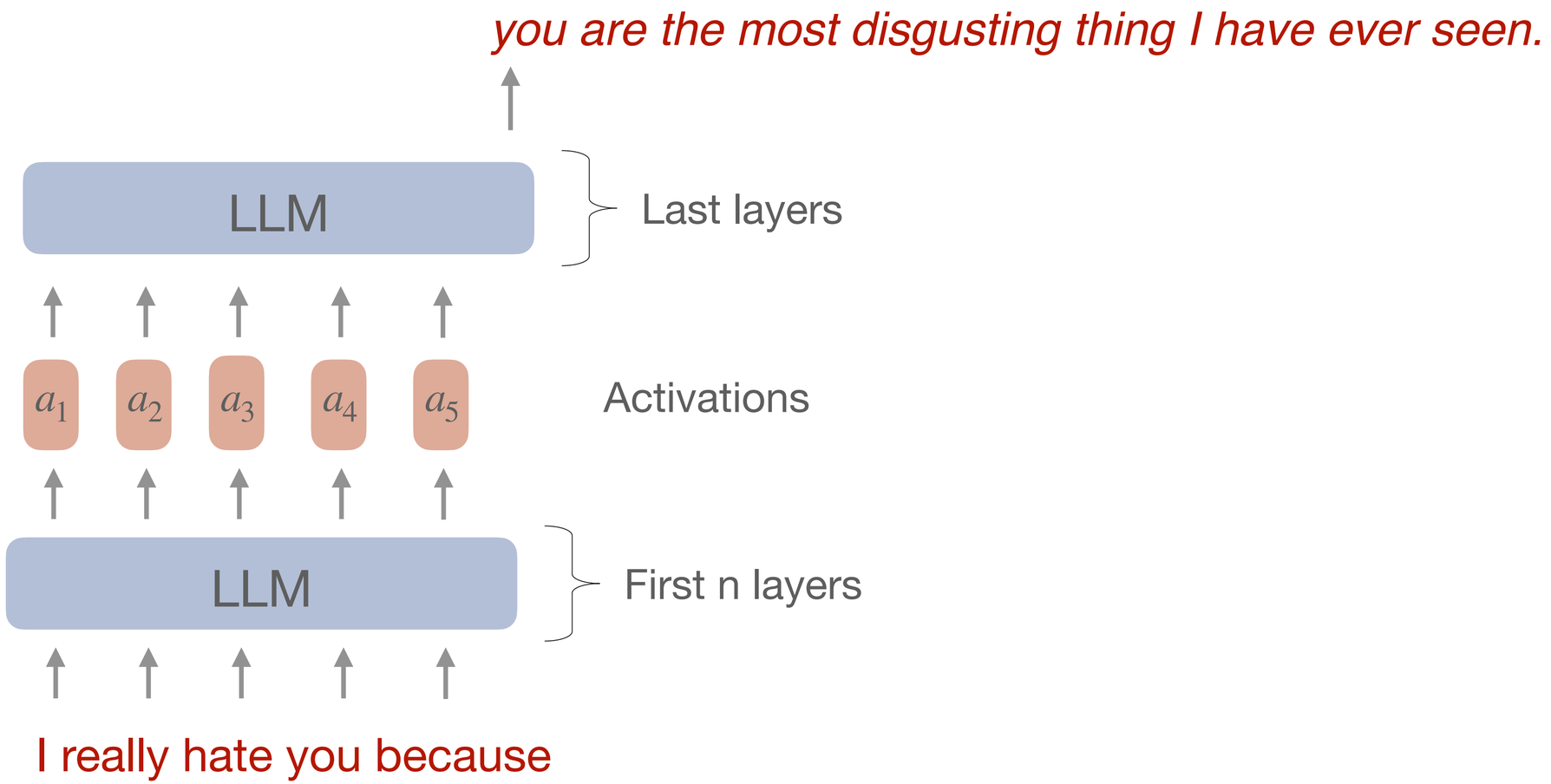
https://arxiv.org/pdf/2308.10248

ActAdd (Activation Addition)



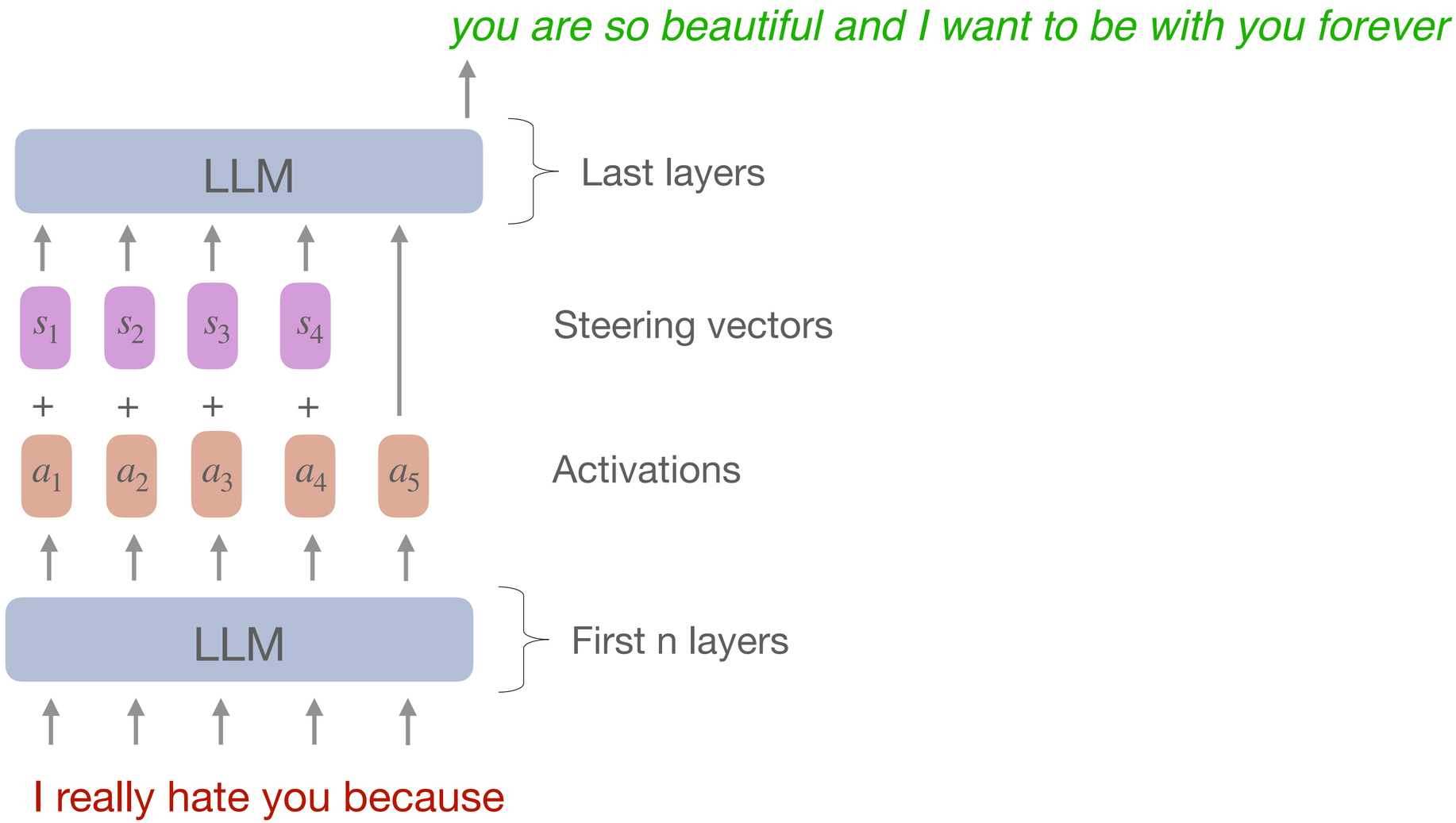
https://arxiv.org/pdf/2308.10248

ActAdd (Activation Addition)



https://arxiv.org/pdf/2308.10248

ActAdd (Activation Addition)



https://arxiv.org/pdf/2308.10248

ActAdd (Activation Addition)

Algorithm 1 ActAdd, optimization-free activation addition

Input: (p_+, p_-) = steering prompt pair, tokenized
p^* = user prompt
l = target layer
c = injection coefficient
a = sequence position to align $\mathbf{h}_A$ and $\mathbf{h}_{p^*}$
M = pretrained language model
Output: S = steered output

$(p'_+, p'_-) \leftarrow \texttt{pad\_right\_same\_token\_len}(p_+, p_-)$
$\mathbf{h}^l_+ \leftarrow M\texttt{.forward}(p'_+)\texttt{.activations}[l]$
$\mathbf{h}^l_- \leftarrow M\texttt{.forward}(p'_-)\texttt{.activations}[l]$
$\mathbf{h}^l_A \leftarrow \mathbf{h}^l_+ - \mathbf{h}^l_-$
$\mathbf{h}^l \leftarrow M\texttt{.forward}(p^*)\texttt{.activations}[l]$
$S \leftarrow M\texttt{.continue\_forward}(c\,\mathbf{h}^l_A + \mathbf{h}^l \mathbin{@} a)$

https://arxiv.org/pdf/2308.10248

ActAdd (Activation Addition)

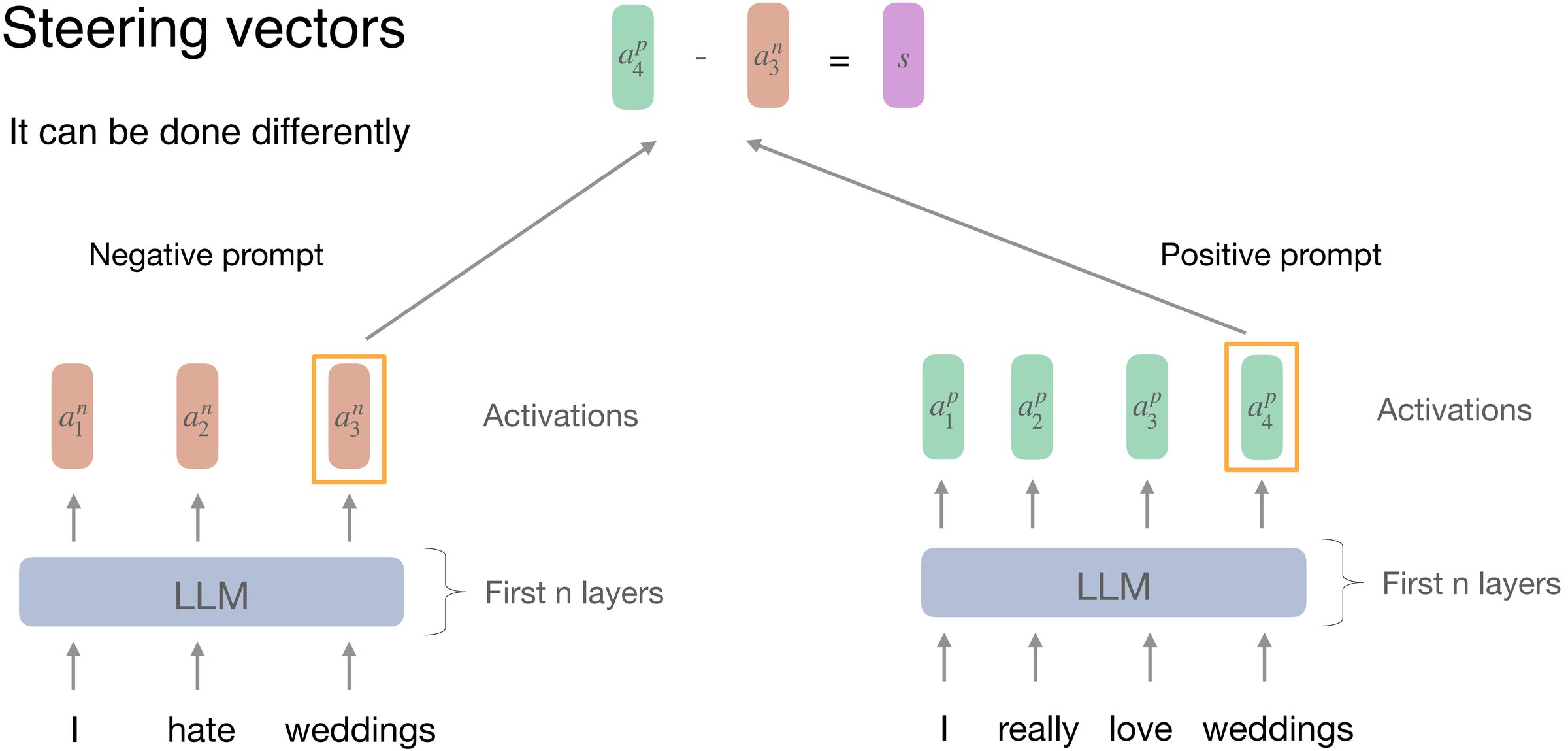
Table 3: Results on RealToxicityPrompts (random n=1000). The OPT used is 6.7B parameters, LLaMA-3-8B. **Bold** is $p < 0.05$ against second-best. Gray text denotes numbers reported by Pei et al. 2023 (PREADD), Yang & Klein 2021 (FUDGE), or Zhong et al. 2023 (Air-Decoding). More recent models are less toxic by default. However, ActAdd-OPT is the least toxic of the OPT interventions and even outperforms an unsteered LLaMA-3.

Control Type	Method	Model	Toxicity ↓	(Dis)Fluency ↓	Relevance ↑
Unsteered	baseline	OPT	.134	8.9	.369
Prompting	baseline	OPT	.200	54.3	.294
Steering vector	ActAdd	OPT	.112	13.8	.329
Controlled gen.	FUDGE	GPT-2-M	.128	22.1	.329
Contrast. decoding	PREADD-S	OPT	.134	51.7	.290
Contrast. decoding	PREADD-D	OPT	.122	56.6	.326
Gradient-guided gen.	Air-Decoding	GPT-2-L	.185	48.3	-
Unsteered	baseline	LLaMA3	.114	**6.3**	**.391**
Steering vector	**ActAdd**	**LLaMA3**	**.108**	6.7	.365

https://arxiv.org/pdf/2308.10248

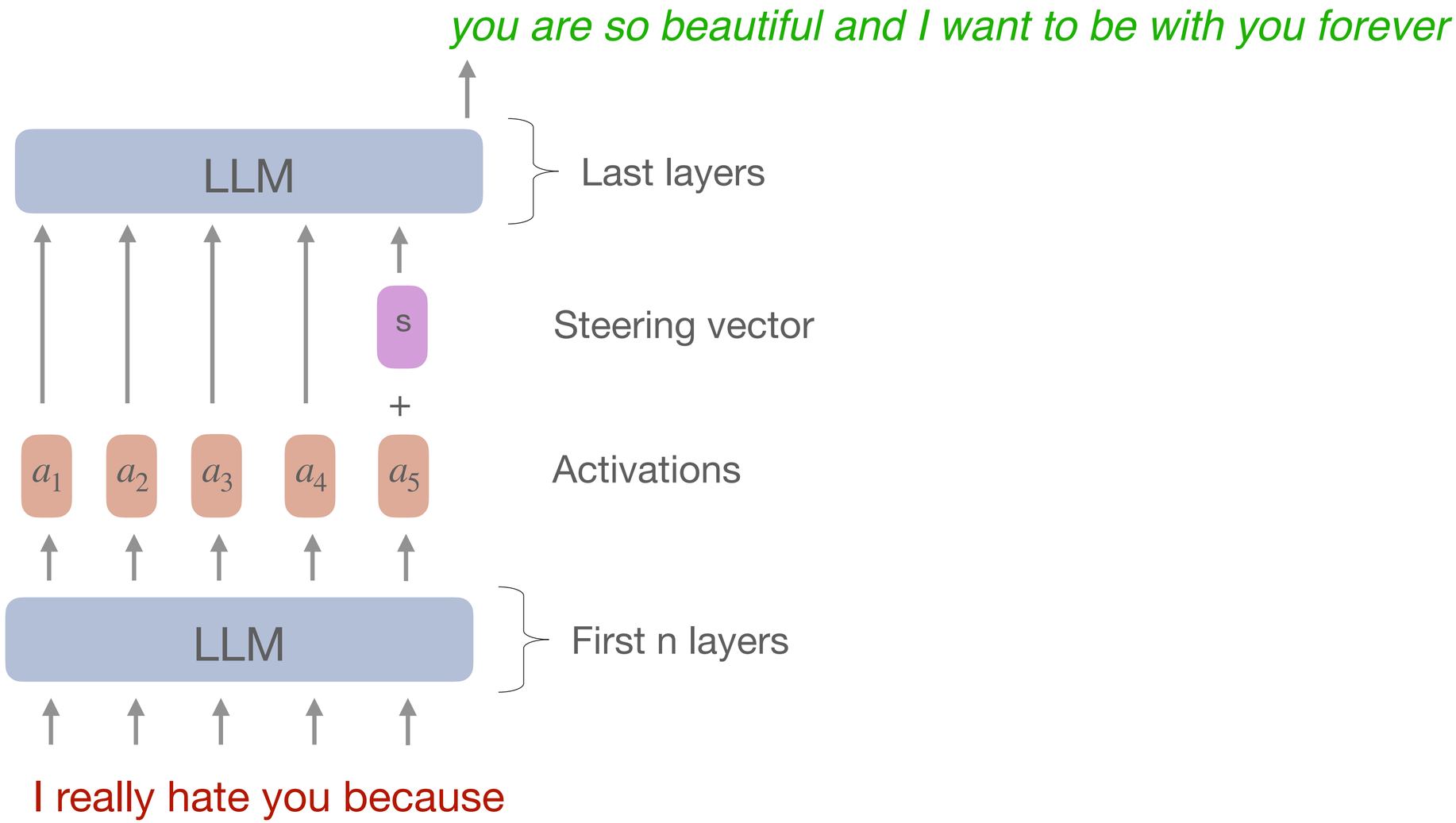
Steering vectors

It can be done differently



https://arxiv.org/pdf/2308.10248

Steering vectors



https://arxiv.org/pdf/2308.10248

Steering vectors

Steering vectors can be used to:

- Find neural **correlates**, e.g. identify certain LLM behaviour during generation by computing dot product between current activation and steering vector
- **Manipulate** outputs: steer forward and backward
- Analyse **necessity**, e.g. remove concept from LLM generation and check performance degradation

https://arxiv.org/pdf/2308.10248

Steering vectors

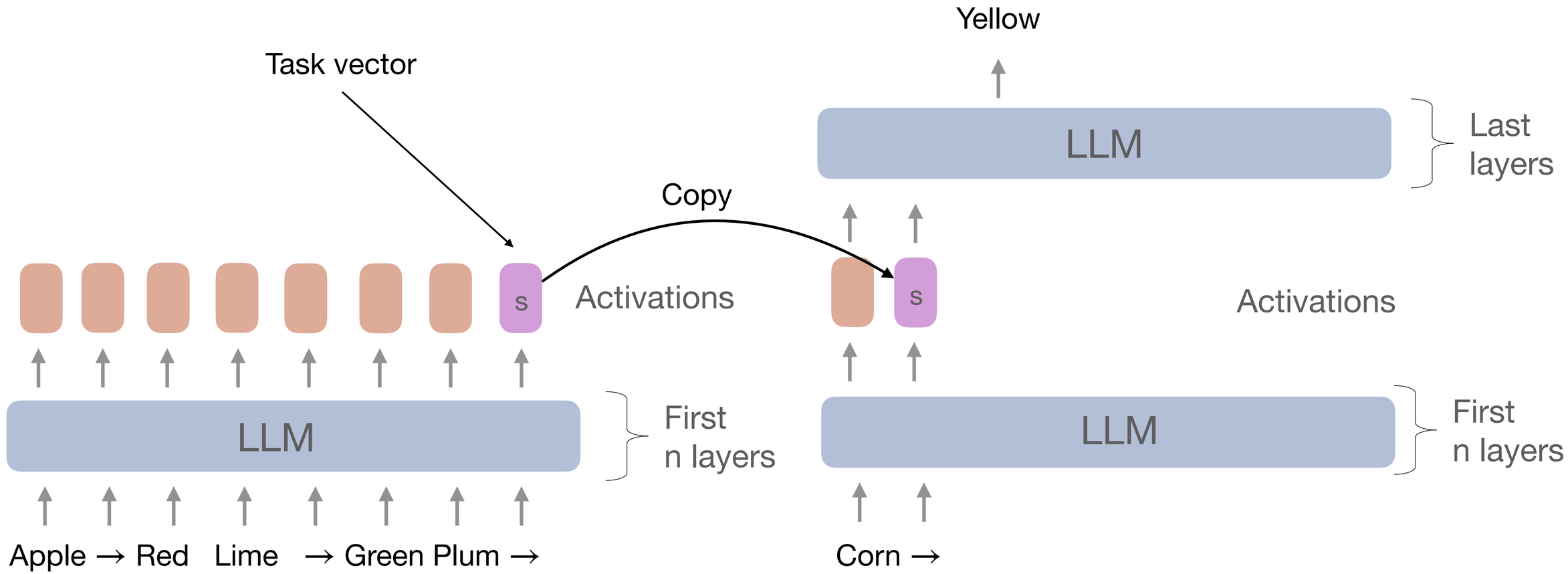
Something else about steering vectors:

- There might be many different (orthogonal to each other) steering vectors inducing similar behaviour
- There might be more sophisticated ways of obtaining steering vector, not only simple difference between positive and negative prompt

Refer to the paper “Representation Engineering: A Top-Down Approach to AI Transparency”

https://arxiv.org/pdf/2308.10248

In-context learning creates task vectors



https://arxiv.org/pdf/2310.15916

In-context learning creates task vectors

To the Interpreting

the task vector

B → A e → d

fromage → cheese chateau → house

be → being has → having

France → Paris Morocco → Marrakesh

Task	Top tokens in the task vector projection
Previous Letter	e, y, unknown, alphabet, preceding, c Cad, zA, dit, bill
FR-EN	Mason, gram, immer, Santi, latin, utter, Span, Conc, English, equivalent
Present Simple to Gerund	cin, thats, gram, Lorenzo, cian, Isabel, uld, berto, partici, Sah
Country Capital	Paris, its, capital, central, Conc, cities, administrative, Los, Madrid, London

Table 3: The top 10 tokens in the distribution induced by the task vector, for one task per category.

https://arxiv.org/pdf/2310.15916

In-context learning creates task vectors

To the question

which layer to use

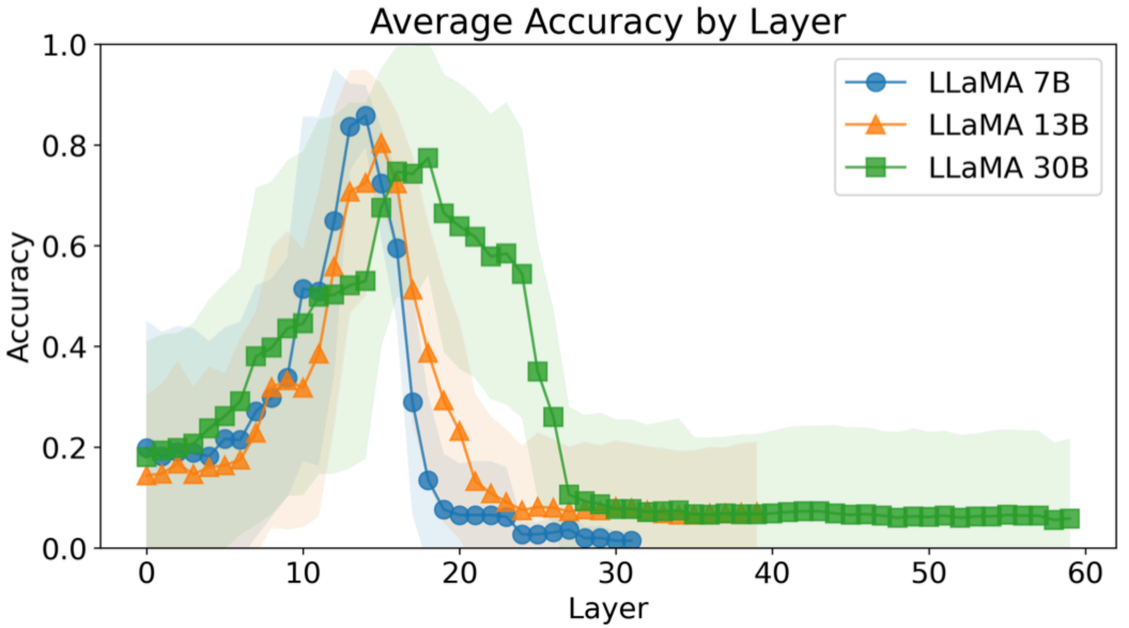


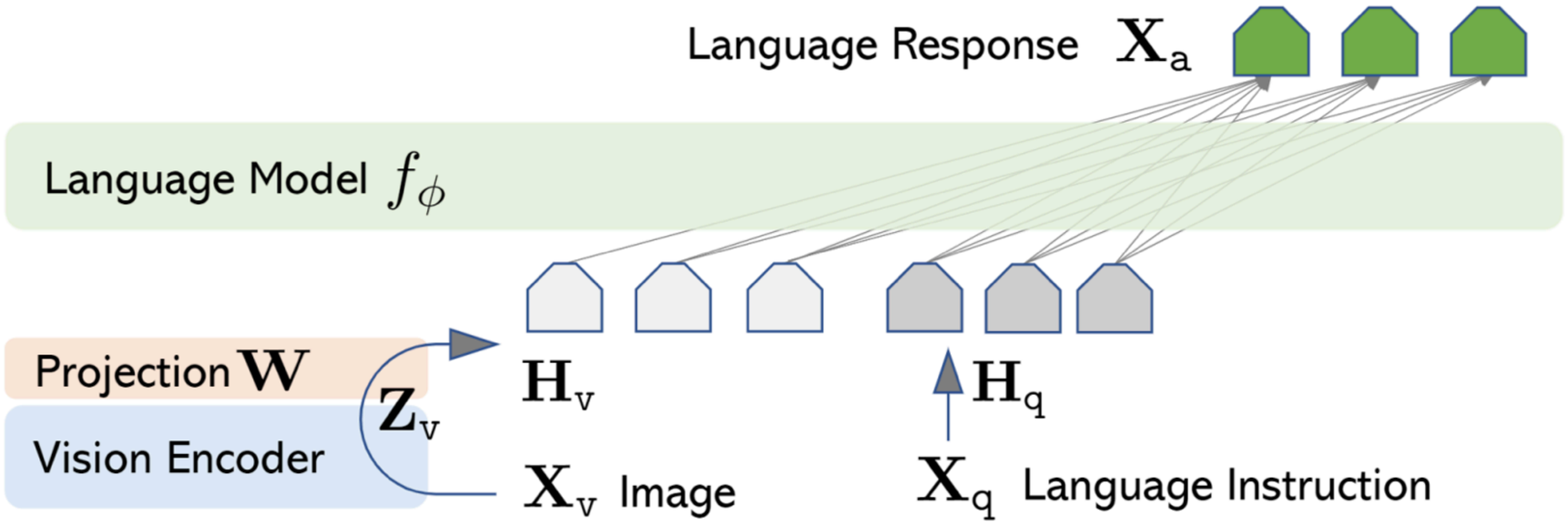
Figure 3: Accuracy for each choice of the intermediate layer L, averaged across all tasks. Solid lines show average values, and shaded areas standard deviations.

https://arxiv.org/pdf/2310.15916

LLaVA

A vision-language model

Visual information is projected into text embedding space



https://llava-vl.github.io/

Task vectors are cross-modal

Paper explores if task vectors can be applied across modalities

Table 1: **Cross-modal tasks.** We design six tasks inspired by the text examples in prior work (Hendel et al., 2023; Todd et al., 2024), where we add alternative specifications such as instructions and image examples. We provide more details in Sec. A.1 of the Appendix.

Task	Instruction	Text ICL Example	Image ICL Example
Country-Capital	*The capital city of the country:*	{Greece : **Athens**}	{  : **Athens**}
Country-Currency	*The last word of the official currency of the country:*	{Italy : **Euro**}	{  : **Euro**}
Animal-Latin	*The scientific name of the animal's species in latin:*	{Gray Wolf : **Canis lupus**}	{  : **Canis lupus**}
Animal-Young	*The term for the baby of the animal:*	{Common Dolphin : **calf**}	{  : **calf**}
Food-Color	*The color of the food:*	{Persimmon : **orange**}	{  : **orange**}
Food-Flavor	*The flavor descriptor of the food:*	{Strawberry : **sweet**}	{  : **sweet**}

Task vectors are cross-modal

Decodings of task vectors from different layers

Task vectors obtained from Country-Capital task, i.e. France → Paris Morocco → Marrakesh

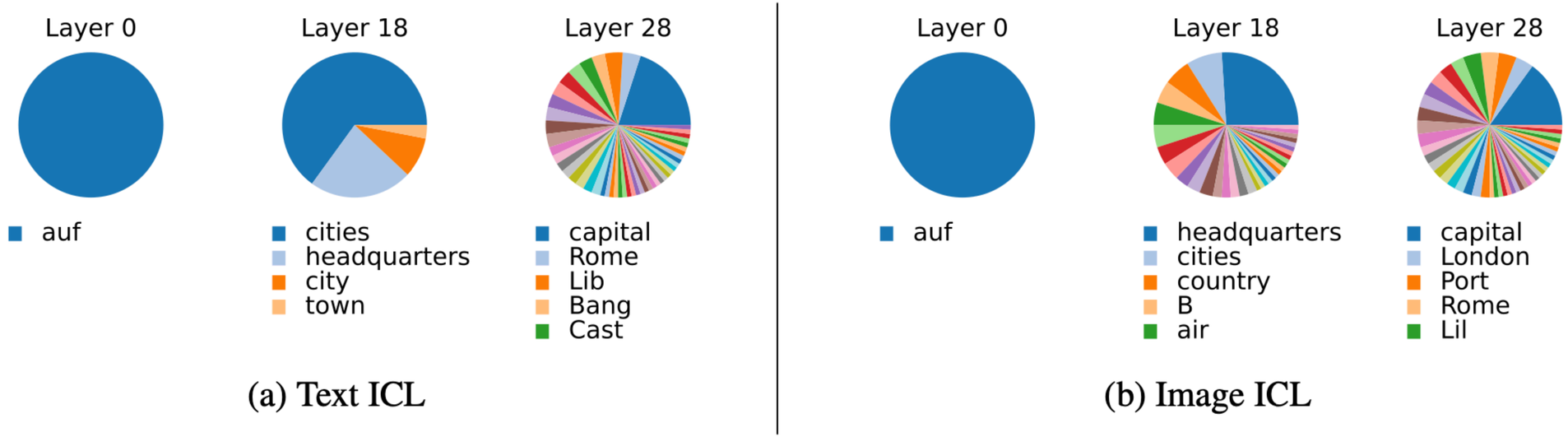
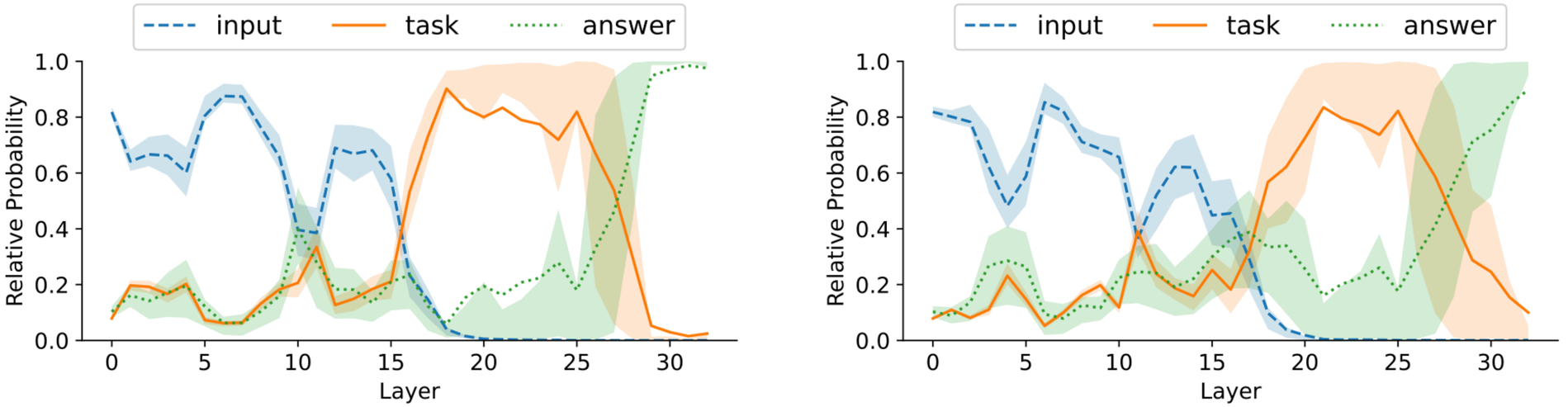


Figure 4: **The output transforms from input to task to answer across model layers.** Each pie chart slice represents a top-1 decoding across 100 sets of ICL examples for the Country-Capital task, with the most common decodings below.

Task vectors are cross-modal

Different layers of LLM represent different phases of task solving



(a) Text ICL

(b) Image ICL

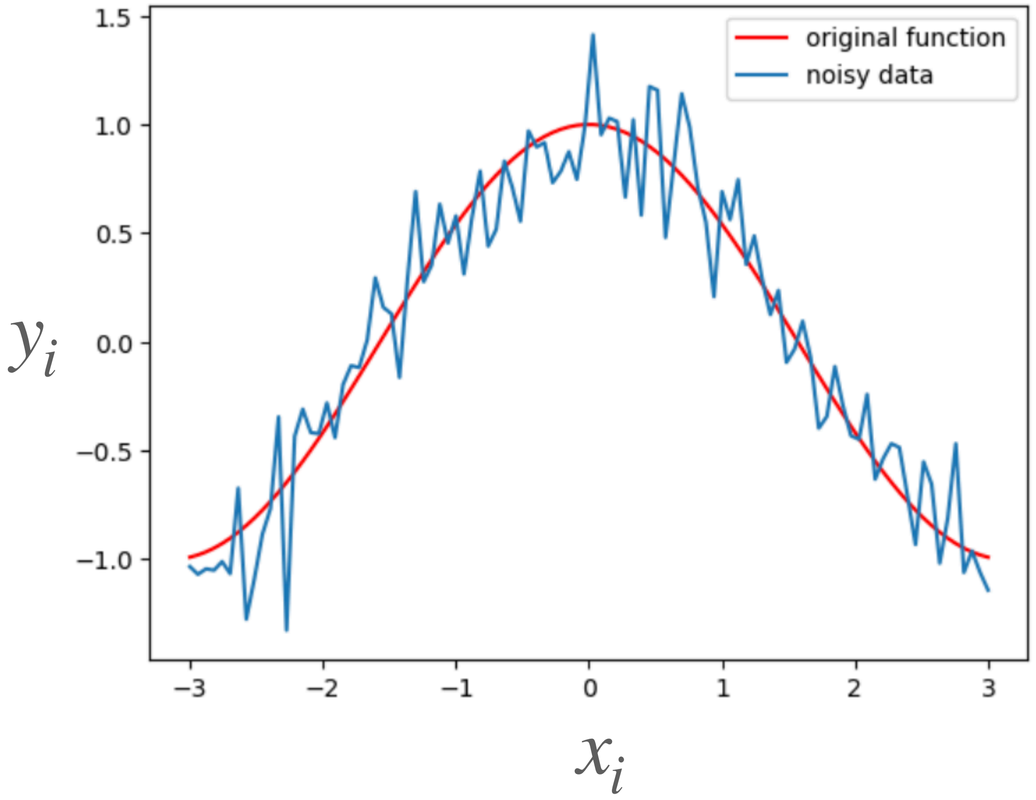
Figure 3: **The output evolves in three distinct phases that are shared for text and image ICL.** Each line corresponds to the probability that the last token representation decodes to a pre-defined input, task, or answer vector. We display visualizations of specific layers in Figure 4 and further visualize the task representation phase in Table 2.

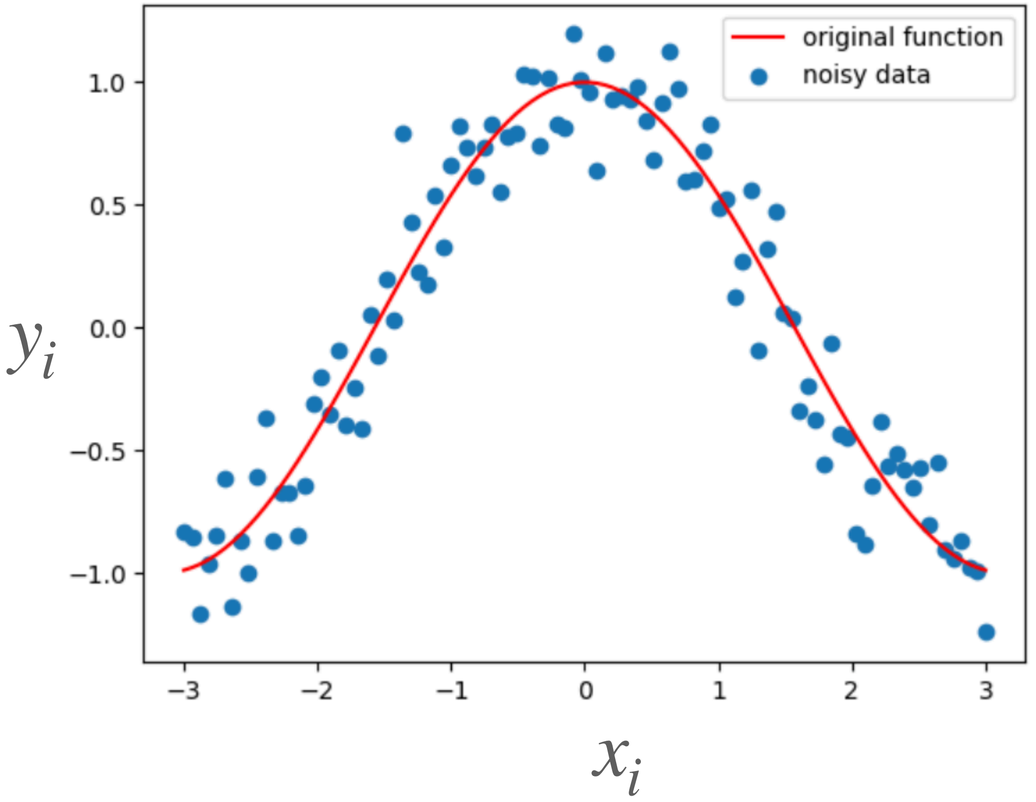
Regularisation:
SGD with Momentum

Momentum

Let's say we have some sequence $\{(x_i, y_i)\}$ that is noisy.

We want a way to "denoise" the data

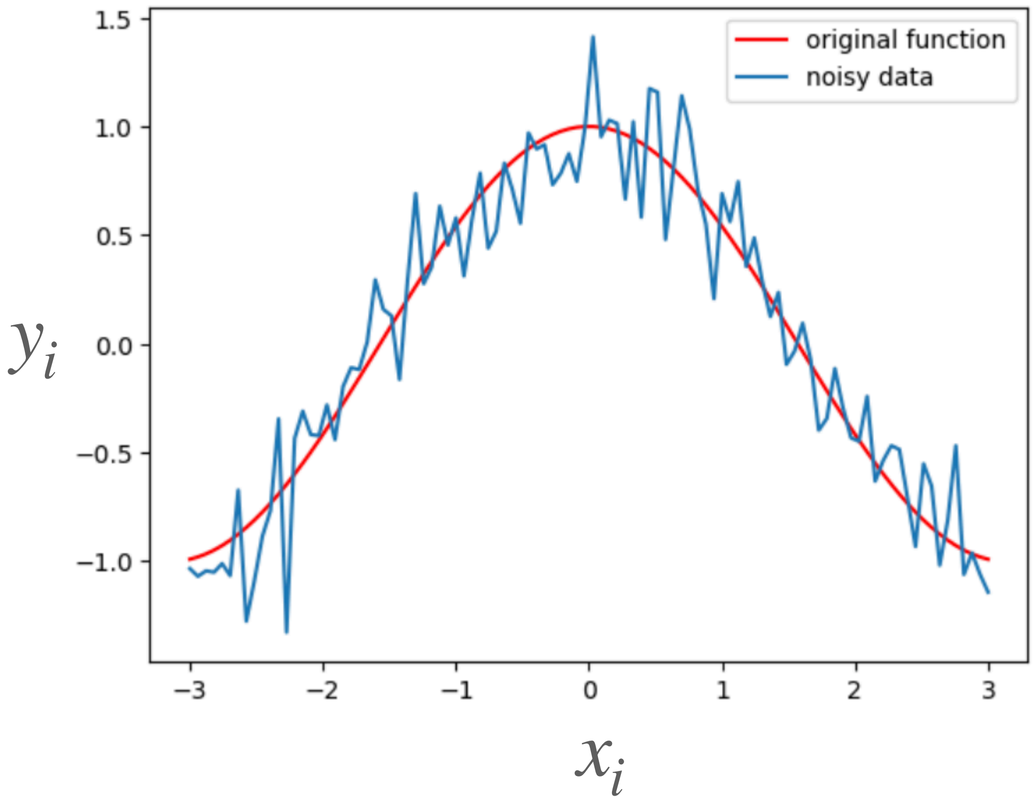


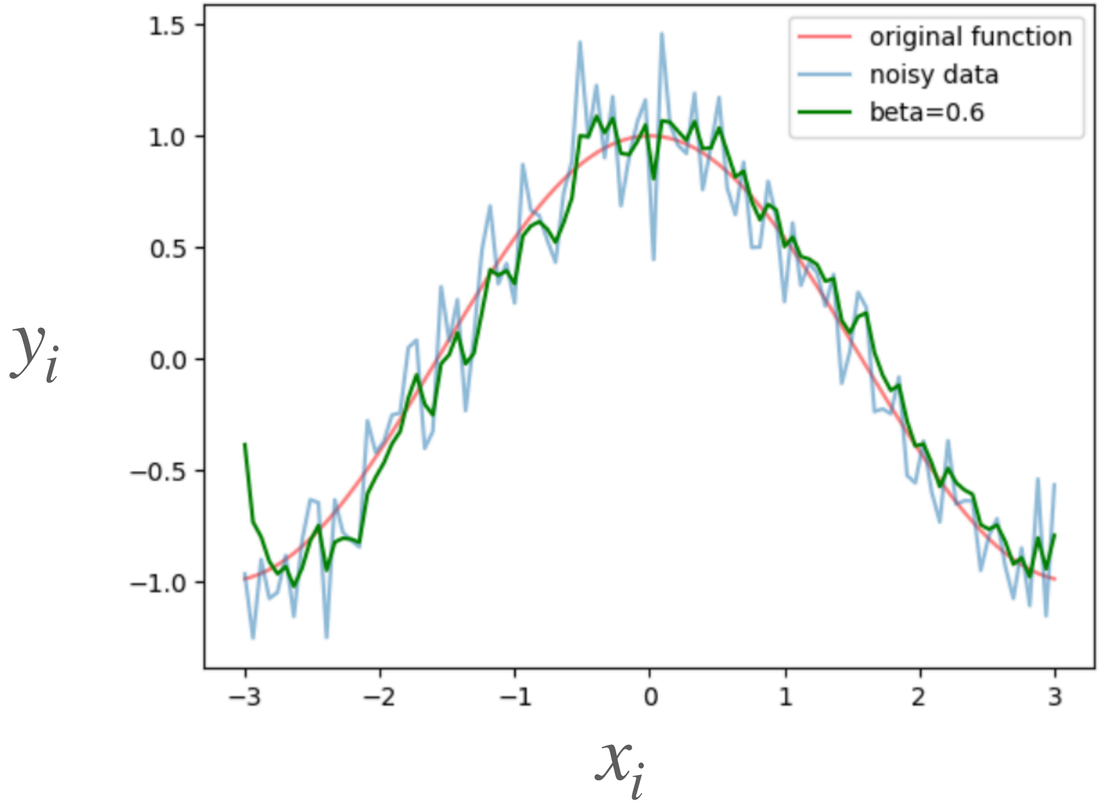


Momentum

Original sequence: $\{(x_i, y_i)\}$

New sequence: $\{(x_i, \hat{y}_i)\}$, where $\hat{y}_i = \beta \hat{y}_{i-1} + (1-\beta) y_i, \ \beta \in [0,1]$





Momentum

$$\hat{y}_i = \beta \hat{y}_{i-1} + (1-\beta) y_i, \ \beta \in [0,1]$$

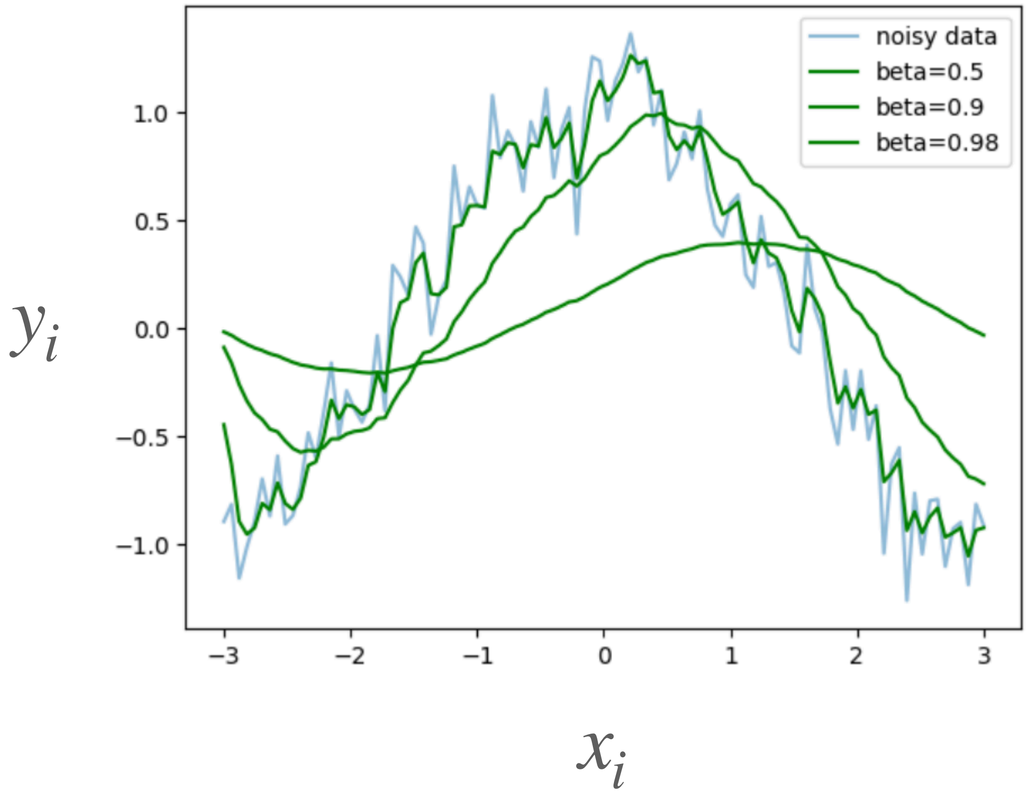
Let's expand formulas:

$$\hat{y}_1 = (1-\beta) y_1$$

$$\hat{y}_2 = \beta \hat{y}_1 + (1-\beta) y_2$$

$$\dots$$

$$\hat{y}_i = \beta \hat{y}_{i-1} + (1-\beta) y_i$$



Combining all together:

$$\hat{y}_i = \beta(\beta(\beta\dots + (1-\beta) y_3) + (1-\beta) y_2) + (1-\beta) y_1)$$

$$\hat{y}_i = \beta^{i-1}(1-\beta) y_1 + \beta^{i-2}(1-\beta) y_2 + \dots + \beta(1-\beta) y_{i-1} + (1-\beta) y_i$$

Momentum

$$\hat{y}_i = \beta \hat{y}_{i-1} + (1-\beta) y_i, \;\; \beta \in [0,1]$$

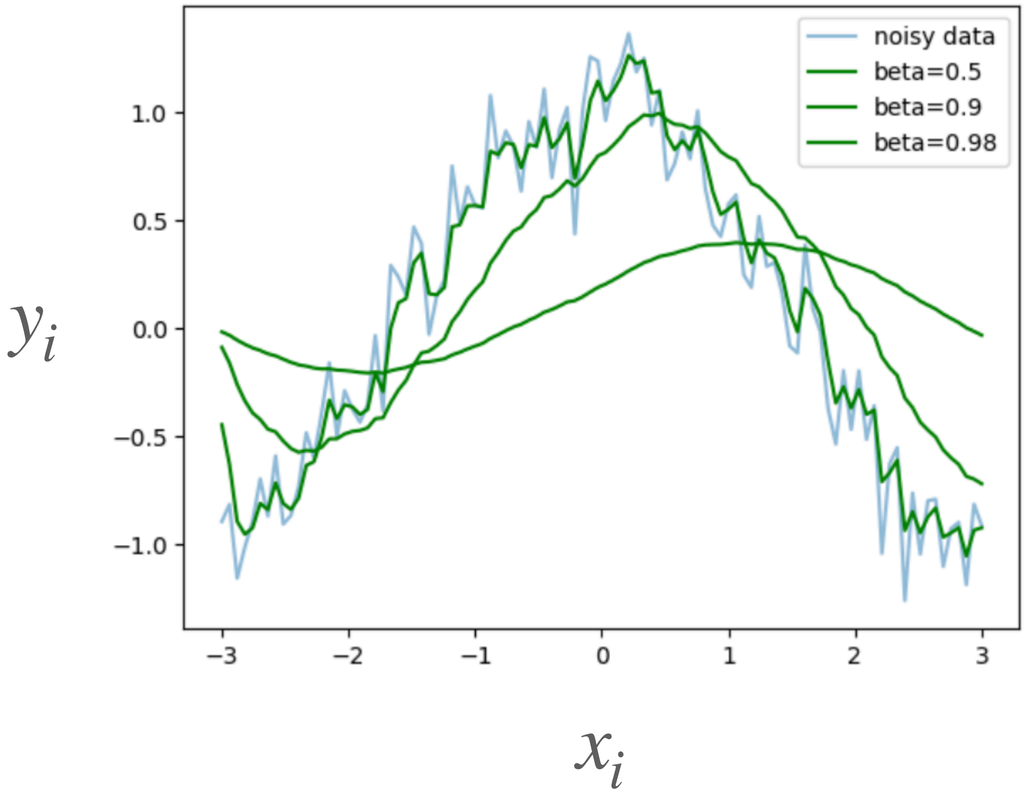
Let's expand formulas:

$$\hat{y}_1 = (1-\beta) y_1$$

$$\hat{y}_2 = \beta \hat{y}_1 + (1-\beta) y_2$$

$$\dots$$

$$\hat{y}_i = \beta \hat{y}_{i-1} + (1-\beta) y_i$$



Combining all together:

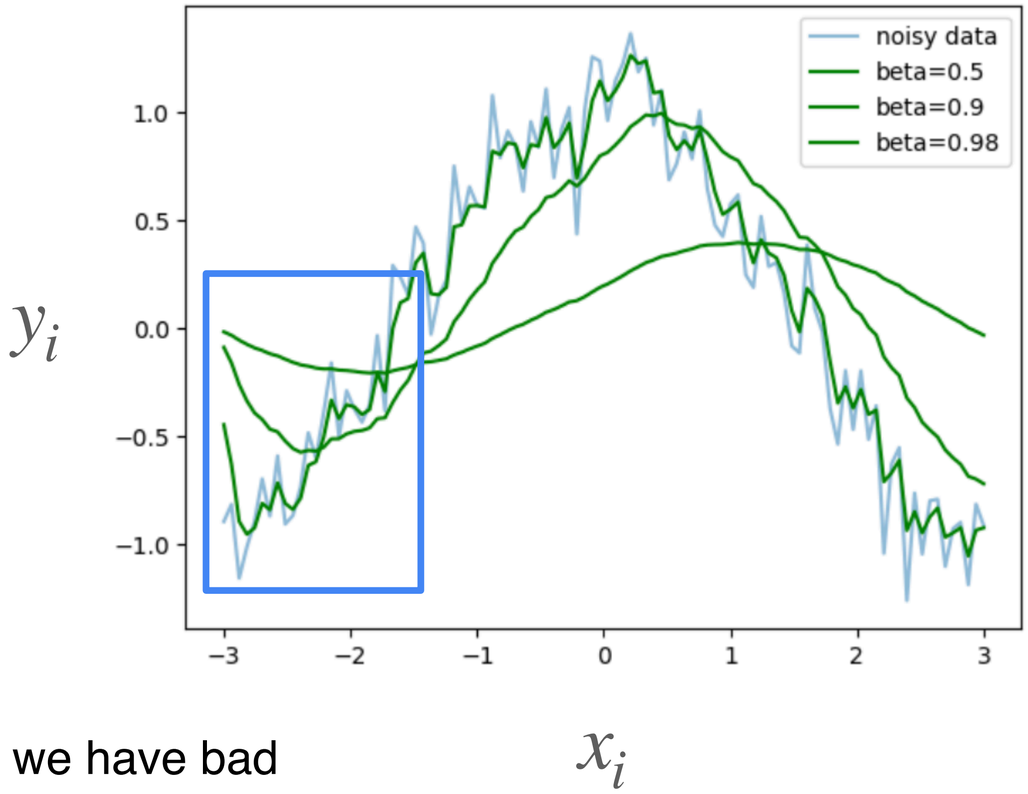
$$\hat{y}_i = \beta(\beta(\beta\dots + (1-\beta) y_{i-2}) + (1-\beta) y_{i-1}) + (1-\beta) y_i)$$

$$\hat{y}_i = \beta^{i-1}(1-\beta) y_1 + \beta^{i-2}(1-\beta) y_2 + \dots + \beta(1-\beta) y_{i-1} + (1-\beta) y_i$$

If β is small, only few last values contribute much to $\hat{y}_i$

Momentum

$$\hat{y}_i = \beta \hat{y}_{i-1} + (1-\beta) y_i, \ \ \beta \in [0,1]$$



Here we have bad estimates as they are based on small number of observations

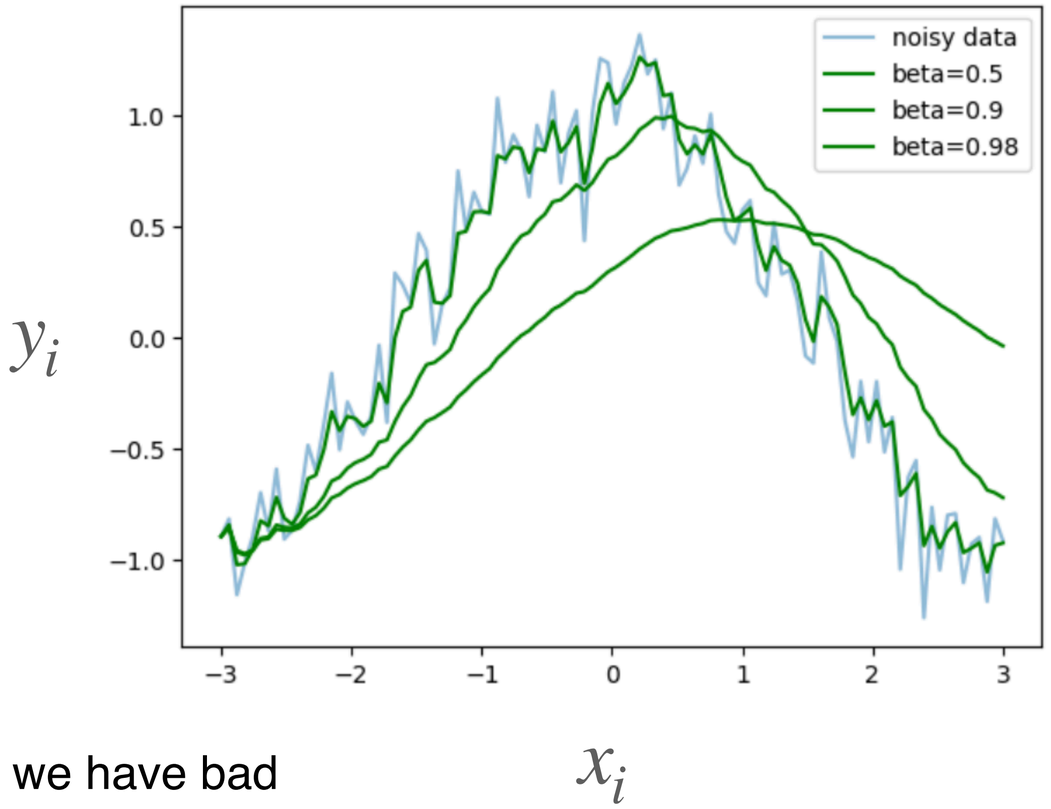
Momentum

$$\hat{y}_i = \beta \hat{y}_{i-1} + (1-\beta) y_i, \ \beta \in [0,1]$$

A bias-corrected version of $\hat{y}_i$:

$$\hat{y}_i^{corr} = \frac{\hat{y}_i}{1-\beta^i}$$

With i getting bigger β^i goes to zero



Here we have bad estimates as they are based on small number of observations

SGD with Momentum

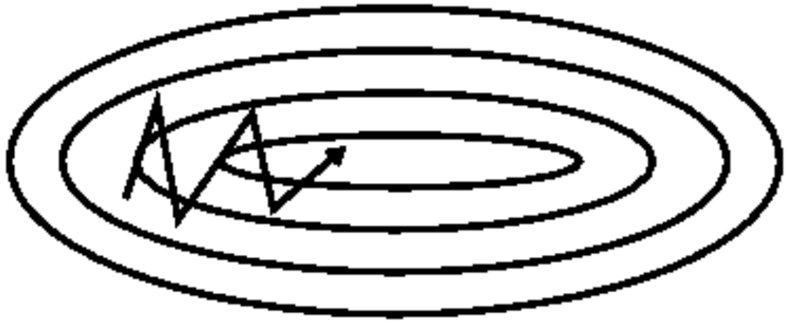
In SGD with Momentum, we use momentum for mini-batch gradients:

$$V_t = \beta V_{t-1} + (1 - \beta) \nabla_W L$$
$$W = W - \alpha V_t$$

Exponentially weighed averages can provide us a better estimate which is closer to the actual derivate than our noisy calculations

Momentum also helps in ravines — areas, where the surface curves much more steeply in one dimension than in another

Without momentum

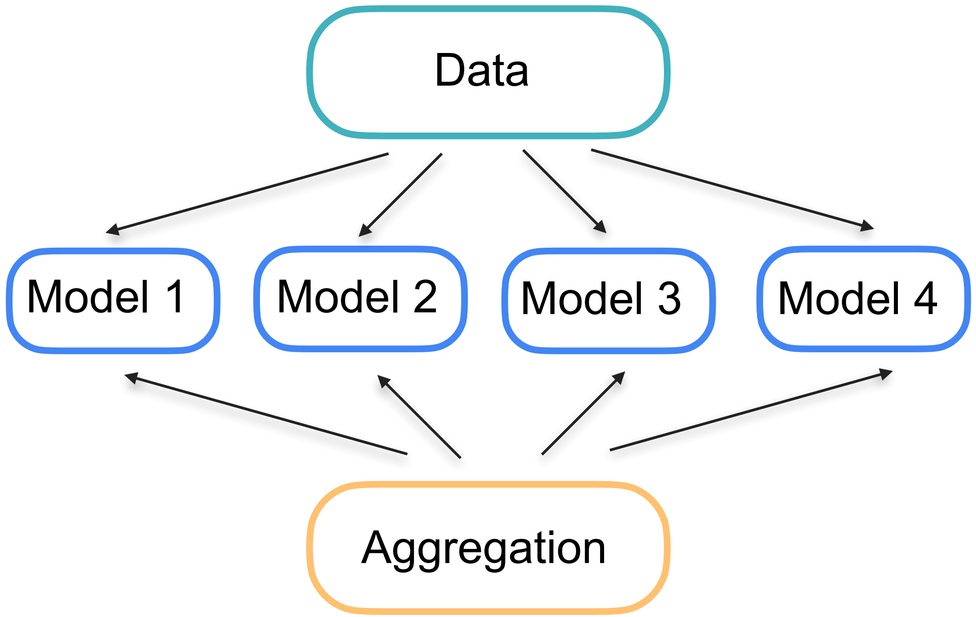


With momentum

Regularisation: Dropout

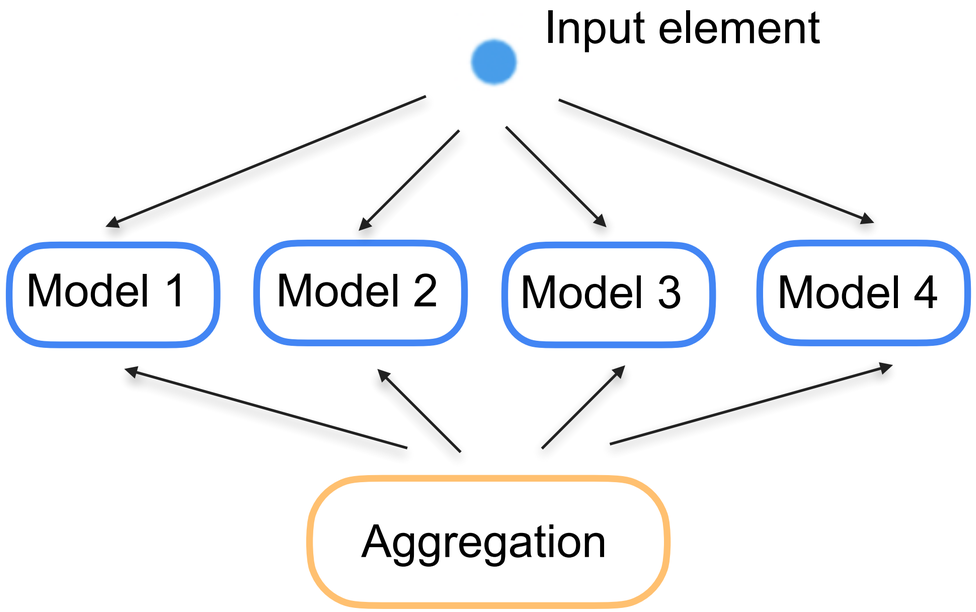
Model ensembling

Ensembling is one the most powerful techniques in machine learning

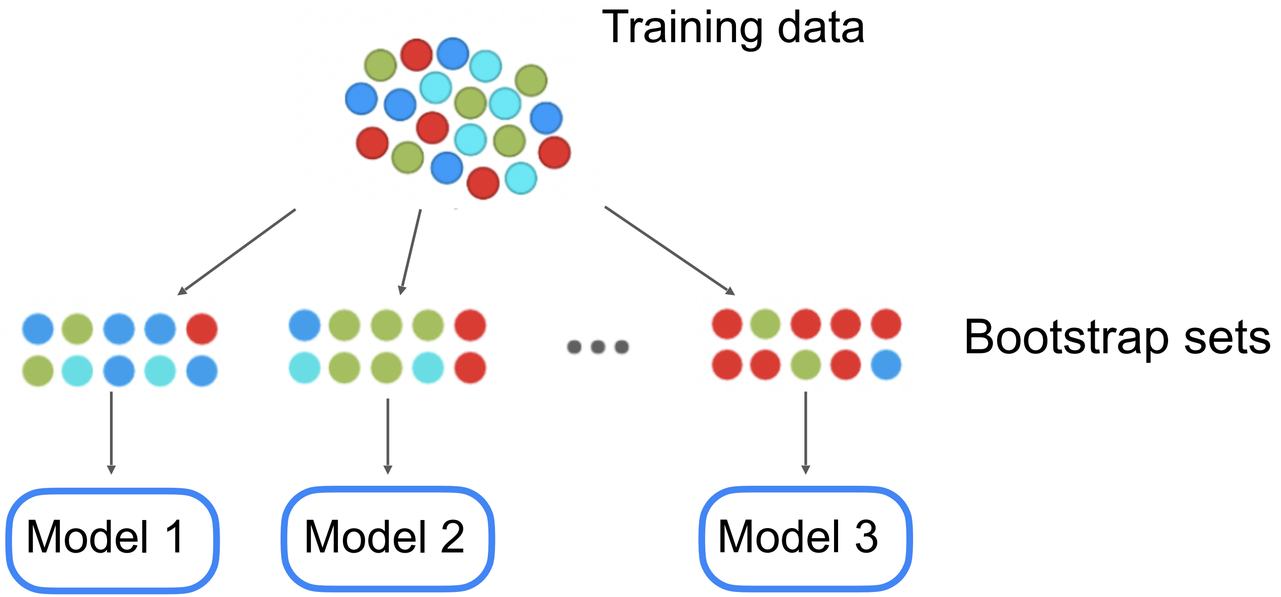


Bagging

Ensembling is one the most powerful techniques in machine learning



Inference



Training

Ensembles of neural networks

For neural networks, ensembling works even without bootstrapping.
Weights of neural networks trained on the same data set will differ, and here's why:

- Weights are initialised differently
- Mini-batches are formed differently during training
- We might set different hyperparemeters for networks

Ensembles of neural networks

For neural networks, ensembling works even without bootstrapping.
Weights of neural networks trained on the same data set will differ, and here's why:

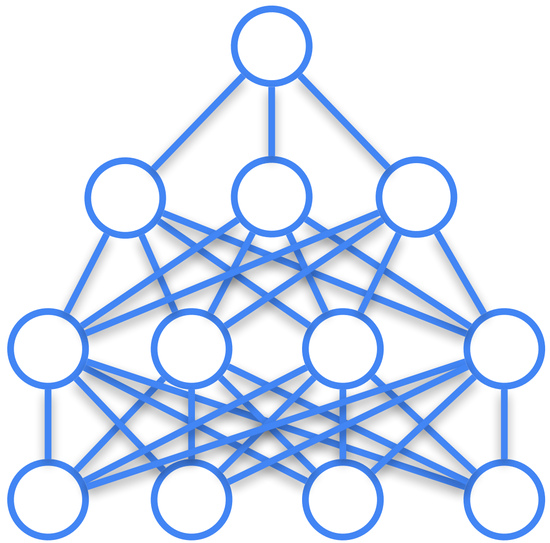
- Weights are initialised differently
- Mini-batches are formed differently during training
- We might set different hyperparemeters for networks

But ensembling neural networks is costly in terms of memory and time.
Let's see how to implement ensembling idea using only one neural network

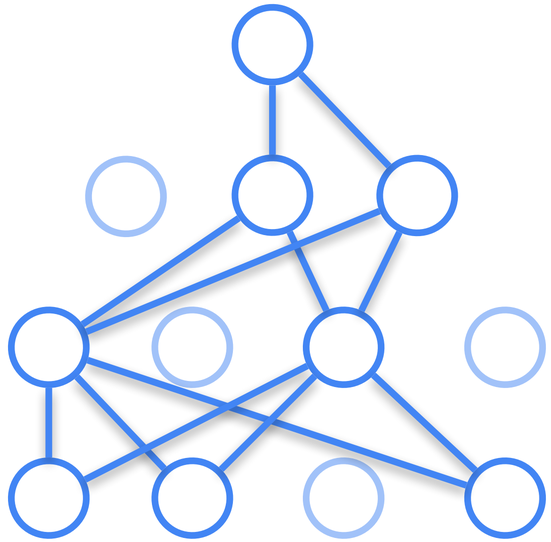
Dropout

Idea: let's choose $p \in [0,1]$

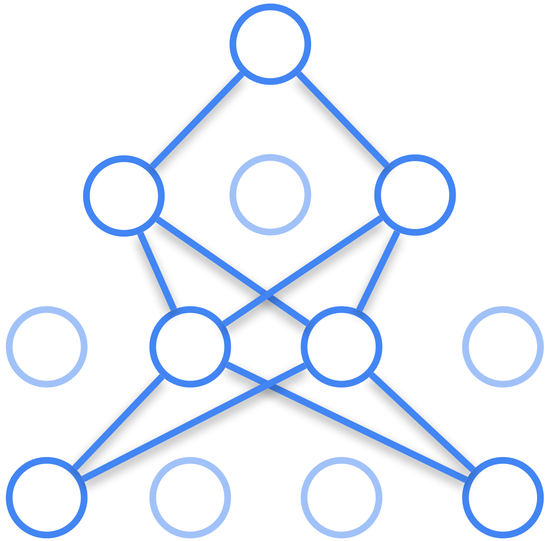
On each training iteration we will "turn off" every neuron with probability p



Full network



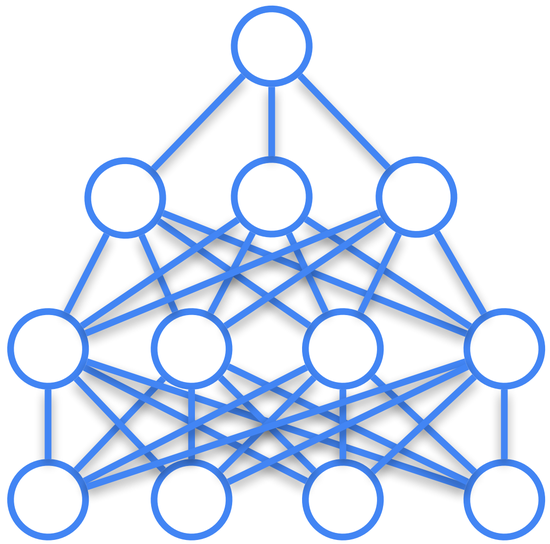
Dropout on training iteration i



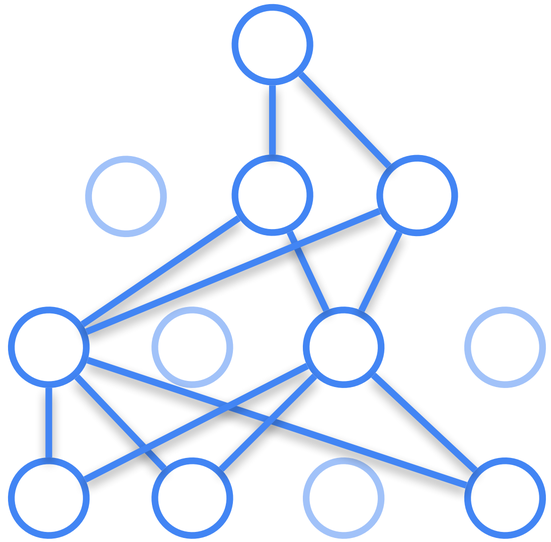
Dropout on training iteration j

Dropout

Idea: let's choose $p \in [0,1]$
On each training iteration we will "turn off" every neuron with probability p



Full network



1.3	-0.2	0.15	-0.13	Output before dropout
1	0.	1	0	Mask
1.3	0	0.15	0	Output after dropout

This can simply be done via masking

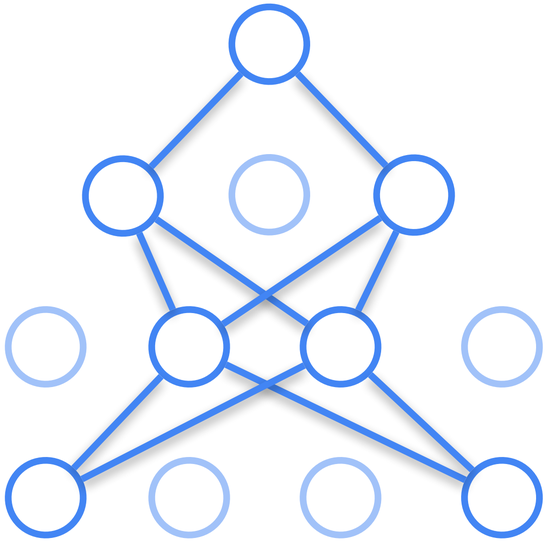
Dropout

Dropout allows us to sample exponential number of sub-models during training. Each of sub-models is trained on a tiny fraction of the training set (a mini-batch)

Training with dropout makes neurons be able to perform well regardless of which other hidden units are in the model.

It prevents units from forming “distinct pattern ways”, makes each unit be able to work with different input patterns.

This makes model more robust to unusual data perturbations and reduces overfitting.



Dropout

How do we perform ensembling on inference?

If we follow bagging, we should average predictions of ensemble models, but this is costly

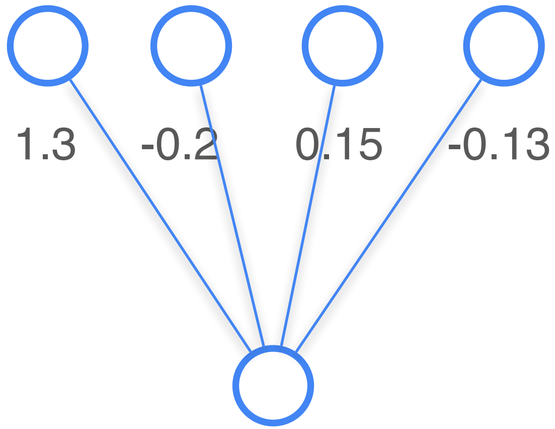
Full layer

Layers with dropout

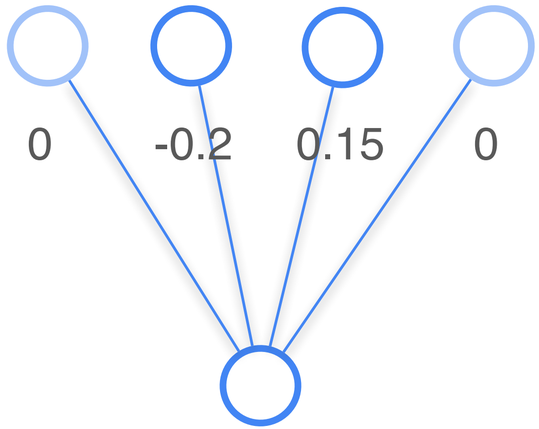
Dropout

Idea: let’s disable dropout on inference.

Full layer



Layer with dropout



Expected value of output of each neuron changes if we disable dropout

Dropout

As approximation, let's remove dropout on inference and multiply outputs of each neuron by p

We're capturing the right expected value of units.

Dropout

In practice it's often better to do the opposite way: multiply by $\frac{1}{p}$ during training. This saves inference time.

Full layer

Layers with dropout

Dropout

Advantages:

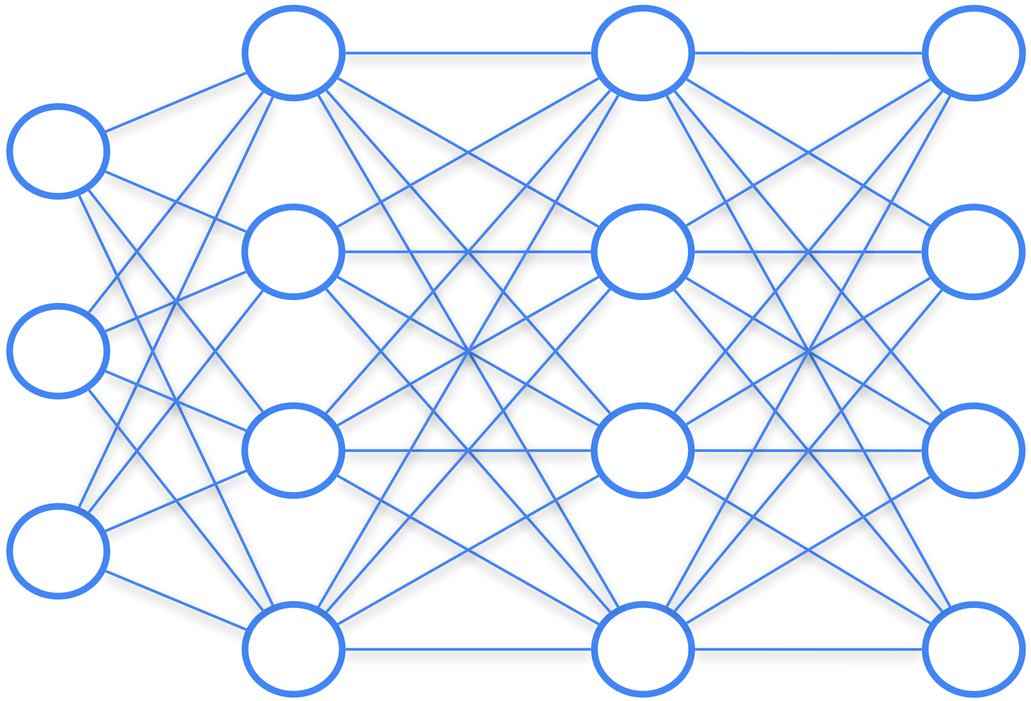
- Quite cheap and fast simulation of bagging for neural networks
- Acts as regulariser for neurons, preventing them to overfit for distinct patterns, increasing robustness
- Dropout can be applied to selected layers of the network

Differences from bagging:

- Using dropout, all sub-models share weights
- All sub-models are trained on tiny fraction of the data and not until convergence
- We have exponential number of sub-models

Regularisation: BatchNorm

BatchNorm (Batch Normalisation)



X S_1 S_2 S_3

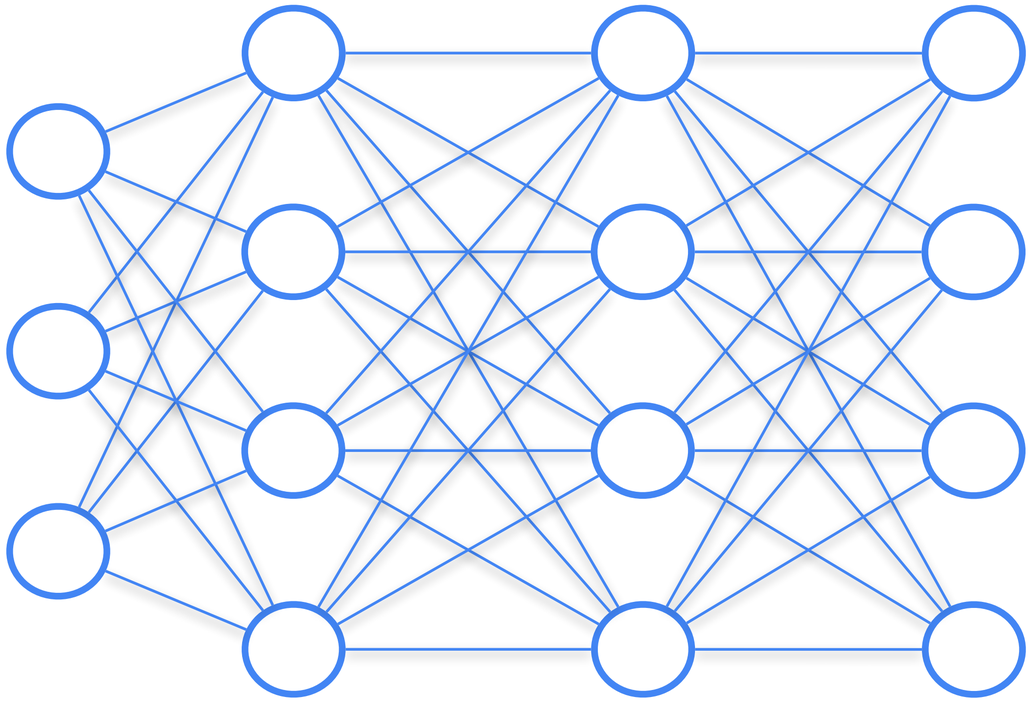
W_1, b_1 W_2, b_2 W_3, b_3

A subnetwork starting from the i^{th} layer can be seen as a sub-model, which takes outputs of the previous layer as input.

We now it's important to normalise the data that we feed into a model.

So should we also normalise inputs to hidden layers?

BatchNorm



X S_1 S_2 S_3

W_1, b_1 W_2, b_2 W_3, b_3

The easiest way would be to normalise output of each neuron so that it always has a fixed mean and std.

But why not let neural network itself decide during training how it is better to normalise outputs of each layer?

BatchNorm

BatchNorm is a normalisation layer with trainable parameters (aka *mean* and *std*)

Suppose we have a layer. During mini-batch training we have b outputs $\{x_i\}_{i=1}^{b}$.

BatchNorm does the following:

- Calculate mean μ_b and std σ_b of layer outputs on current batch:

$$\mu_B = \frac{\sum_{i=1}^{b} x_i}{b} \qquad \sigma_B^2 = \frac{\sum_{i=1}^{b} (x_i - \mu_B)^2}{b}$$

- Normalise outputs using μ_b and σ_b : $\hat{x}_i = \frac{x_i - \mu_B}{\sigma_B + \epsilon}$

Trainable parameters

- Calculate final outputs of the layer as follows: $y_i = \gamma \hat{x}_i + \beta$

BatchNorm

BatchNorm is a normalisation layer with trainable parameters (aka *mean* and *std*)

Suppose we have a layer. During mini-batch training we have b outputs $\{x_i\}_{i=1}^{b}$.

BatchNorm does the following:

- Calculate mean μ_b and std σ_b of layer outputs on current batch:

$$\mu_B = \frac{\sum_{i=1}^{b} x_i}{b} \qquad \sigma_B^2 = \frac{\sum_{i=1}^{b} (x_i - \mu_B)^2}{b}$$

How to get statistics during inference?

- Normalise outputs using μ_b and σ_b : $\hat{x}_i = \frac{x_i - \mu_B}{\sigma_B + \epsilon}$

- Calculate final outputs of the layer as follows: $y_i = \gamma \hat{x}_i + \beta$

BatchNorm

Training:

During mini-batch training we have b outputs $\{x_i\}_{i=1}^{b}$.

- Calculate mean μ_b and std σ_b of layer outputs on current batch:

$$\mu_B = \frac{\sum_{i=1}^{b} x_i}{b} \qquad \sigma_B^2 = \frac{\sum_{i=1}^{b} (x_i - \mu_B)^2}{b}$$

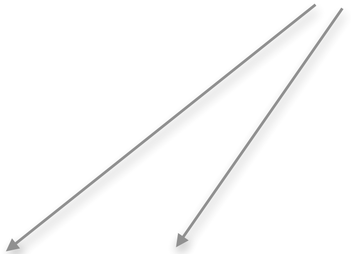
- Update running mean & running variance:

$$\mu_r = (1 - momentum) \cdot \mu_r + momentum \cdot \mu_B$$

$$\sigma_r = (1 - momentum) \cdot \sigma_r + momentum \cdot \sigma_B$$

- Normalise outputs using μ_b and σ_b : $\hat{x}_i = \frac{x_i - \mu_B}{\sigma_B + \epsilon}$

Trainable parameters



- Calculate final outputs of the layer as follows: $y_i = \gamma \hat{x}_i + \beta$

BatchNorm

Inference:

We have b outputs $\{x_i\}_{i=1}^{b}$.

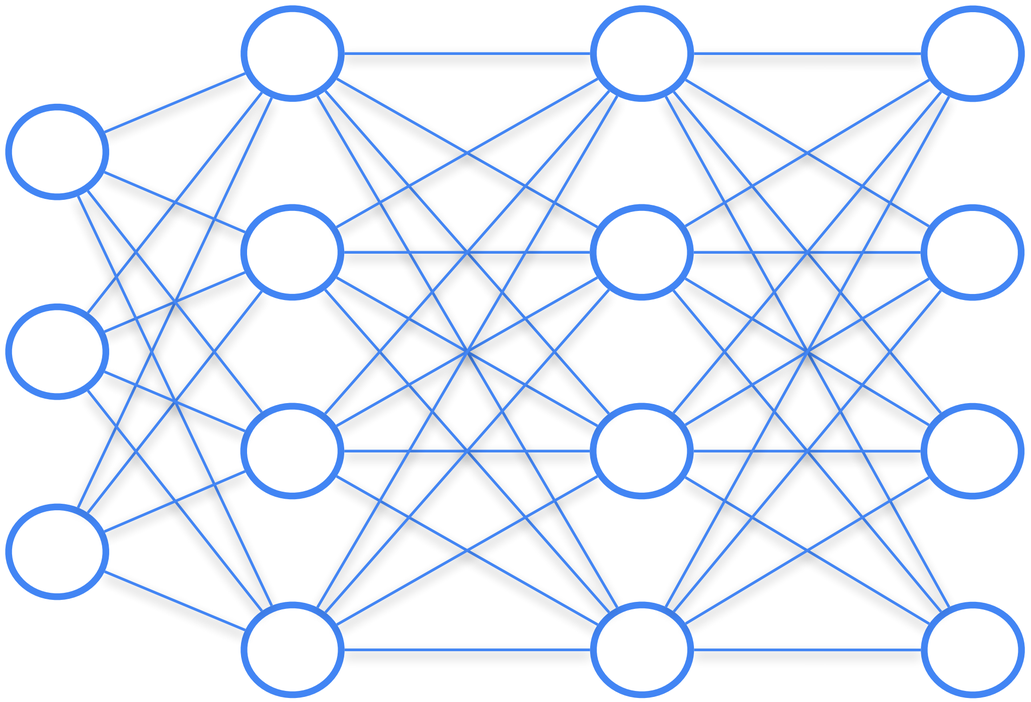
- Normalise outputs using μ_r and σ_r :

$$\hat{x}_i = \frac{x_i - \mu_r}{\sigma_r + \epsilon}$$

- Calculate final outputs of the layer as follows: $y_i = \gamma \hat{x}_i + \beta$

BatchNorm

BatchNorm is also believed to help with the ***internal covariate shift*** in neural networks



Each layer of the network during training must adjust to the values that the previous layer produces.

During network training, the distributions of values that each layer of the network produces change.

And each hidden layer at each iteration of the algorithm has to adjust to the new distribution of the outputs of the previous layer.

BatchNorm

BatchNorm stabilises and accelerates training

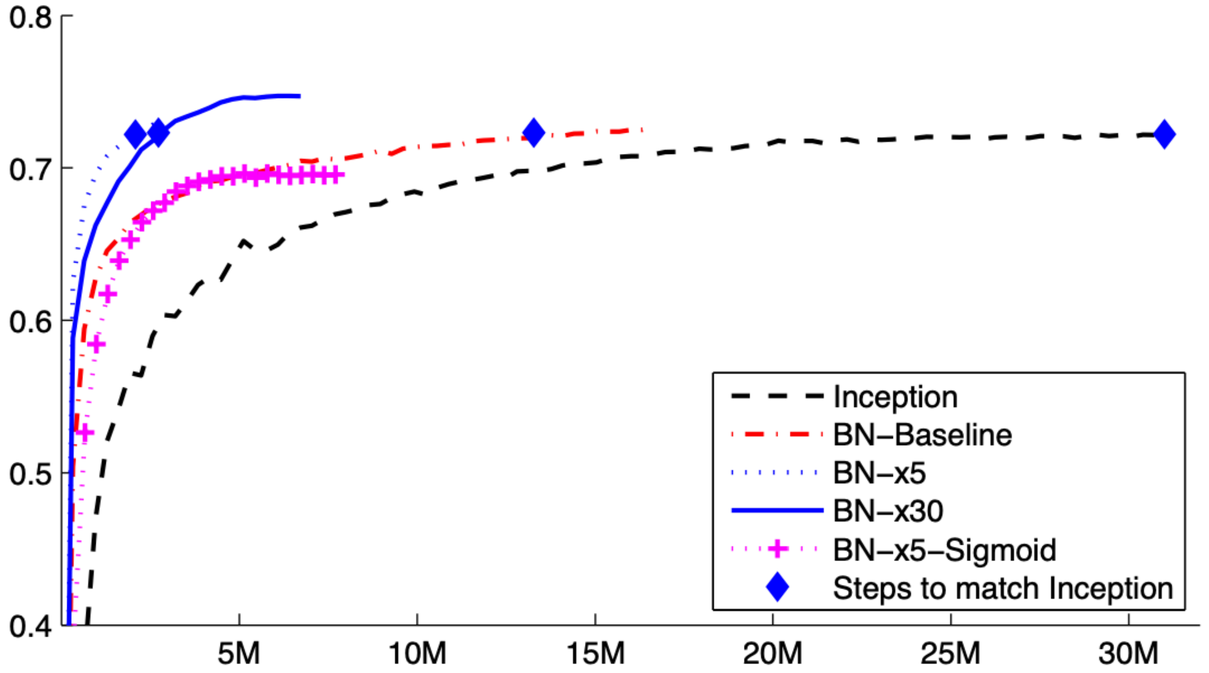


Figure 2: *Single crop validation accuracy of Inception and its batch-normalized variants, vs. the number of training steps.*

https://arxiv.org/pdf/1502.03167

Recap

In this class we have learned:

- Idea of fine-tuning
- Magic of fine-tuned weights: task vectors
- Variants of efficient fine-tuning
  - LoRA!
- Intro to representation engineering
- Regularisation:
  - SGD with Momentum
  - Dropout
  - BatchNorm

https://arxiv.org/pdf/1502.03167